

Rocon Client Service API

사용 설명서

Revision: V1.3.0

Date: 2025/08

개정 이력

버전	작성일	변경 내용
V1.0	2021-08-24	신규 작성
V1.1	2022-02-18	“Rocon Client Server Websocket 접속 주소” 설명 추가 일부 설명 및 예제 코드 보정
V1.1.7	2022-03-31	신규 API 추가 - CMD_GET_RECENT_FAILED_JOB - CMD_RUN_RECENT_FAILED_JOB - CMD_NOTIFY_USER_EVENT - CMD_GET_USER_EVENT - CMD_GET_GPIO - CMD_SET_D_OUT - CMD_SET_A_OUT - CMD_GET_D_IN - CMD_GET_A_IN - CMD_D_IN_LISTENER - CMD_A_IN_LISTENER
V1.1.8	2022-04-14	신규 API 추가 CMD_RESUME_FOR_SUSPEND 4.5 Worker의 Status 세부 설명“에 ‘suspend’, ‘robot_locked’, ‘push_cancel’, ‘required_charging’ 상태 설명 추가함.
V1.2.0	2022-08-10	신규 API 추가 - CMD_GET_WARNING_NOTICE - CMD_SET_WARNING_NOTICE - CMD_CLEAR_WARNING_NOTICE - CMD_GET_ERROR_NOTICE - CMD_SET_ERROR_NOTICE - CMD_CLEAR_ERROR_NOTICE
V1.2.1	2022-09-28	1.4 개발자 지원 항목 추가. 4.4 Feedback Message 항목 내용을 업데이트함. > status 개념 확장 변경됨. 4.5 Worker Status 세부 설명 항목 내용을 업데이트함. > status 개념 확장 변경됨. CMD_LOGIN params 내용 변경 login_id 는 worker의 name으로 변경됨.

V1.2.2	2023-02-27	5.3 change_mode 내용 변경 > navigate 모드 삭제됨. (service 모드로 통합됨)
V1.2.3	2023-04-25	신규 API 추가 - CMD_CHARGE - CMD_PARK - CMD_UNPARK 추가 변경 API - CMD_CREATE_JOB - CMD_GET_RESOURCES 삭제 API - CMD_DOCK (CMD_CHARGE로 대체 요함)
V1.2.4	2023-07-07	신규 API 추가 (Virtual Joystick 전용 API) - CMD_VJ_START - CMD_VJ_FINISH - CMD_VJ_HEALTH_CHECK - CMD_VJ_EVENT
V1.2.5	2023-11-27	신규 API 추가 CMD_MOVE_EXT
V1.2.6	2024-04-05	신규 API 추가 - CMD_DOCK 삭제 API - CMD_CHARGE (CMD_DOCK으로 대체 요함)
V1.2.7	2025-01-10	API 변경 - CMD_DOWNLOAD_MAP_IMAGE ■ Cmd_uuid_size, Cmd_uuid (추가) 설명 변경 - Create job 항목에 charge deprecation 추가 ■ Dock 명령의 arg resource_id 추가
V1.2.8 ~ v1.2.9		V1.2.8 ~ v1.2.9 : 내부 개발 우선 테스트로 내용 기술하지 않음.
V1.3.0	2025-08-19	신규 API 추가 CMD_GET_TASK_DEFINITION (1.18)

(RC v1.7.0 부터 적용)		• CMD_COMPOSE_JOB (1.19) 삭제 API • CMD_CREATE_JOB ■ 공식적인 외부 API로는 더 이상 지원되지 않습니다. ■ CMD_COMPOSE_JOB을 대신 사용.

목차

1. 개요	9
1.1. 주요 키워드 설명	9
1.2. 시스템 구성	10
1.3. 버전 정보	10
1.4. 개발자 지원	10
2. Rocon Client Service 구조와 Rocon Client Service API 설명	12
2.1. 개발자 지원	12
3. API Message 구조	15
3.1. JSON	15
3.2. 필수 Message 필드	15
4. API message 유형	18
4.1. Command Message 구조	18
4.2. Ack Message 구조	19
4.3. Result Message 구조	21
4.4. Feedback Message 구조	23
4.5. Worker의 Status 세부 설명	37
4.6. 모드 별 사용 가능한 Message	43
5. Message 유형별 세부 설명(Custom Client to Rocon Client Service)	45
5.	45
5.1. 로그인: login(Command)	45
5.2. 로그아웃: logout(Command)	47
5.3. 모드변경: change_mode(Command)	48
5.4. 피드백설정: set_feedback(Command)	50
5.5. 리소스정보 요청: get_resources(Command)	51
5.6. Task 정의 요청: get_task_definition(Command)	67

5.7. Job생성: compose_job(Command).....	68
5.8. Job취소: cancel_job(Command)	70
5.9. Job일시정지: pause_job(Command)	71
5.10. Job재개: resume_job(Command).....	72
5.11. 프리 셋 조회: get_presets(Command).....	74
5.12. 프리셋 실행: run_preset(Command)	79
5.13. 이동: move(Command).....	80
5.14. 이동 확장: move_ext(Command)	82
5.15. 멈춤: stop(Command)	84
5.16. 일시정지: pause(Command)	85
5.17. 재개: resume(Command)	86
5.18. 충전: dock(Command) - Service API v1.4.0 부터 신규 추가.....	87
5.19. 맵 다운로드: download_map_image(Command)	89
5.20. 맵 정보 요청: get_map_info(Command)	92
5.21. 지도 변경 요청: change_map(Command).....	94
5.22. 위치 보정 요청: relocation(Command)	97
5.23. 장소 인식 요청: set_position(Command).....	98
5.24. 장애물 정보 제거 요청: clear_obstacle(Command)	99
5.25. 최근 실패한 job 확인: get_recent_failed_job(Command)	100
5.26. 최근 실패한 job 실행: run_recent_failed_job(Command)	101
5.27. 사용자 지정 이벤트 해제: notify_user_event(Command).....	102
5.28. 사용자 지정 이벤트 보기: get_user_event(Command).....	103
5.29. GPIO의 Input/Output값 확인: get_gpio(Command)	103
5.30. GPIO의 digital output값 설정: set_d_out(Command).....	104
5.31. GPIO의 analog output값 설정: set_a_out(Command)	105
5.32. GPIO의 digital input값 등록: get_d_in(Command)	106

5.33. GPIO의 analog input값 등록: get_a_in(Command).....	107
5.34. GPIO의 Digital input 입력 대기: d_in_listener(Command).....	108
5.35. GPIO의 Analog input 입력 대기: a_in_listener(Command).....	109
5.36. Suspend 해제: resume_for_suspend(Command)	110
5.37. Warning Notice 얻기: get_warning_notice(Command).....	111
5.38. Warning Notice 설정: set_warning_notice(Command).....	112
5.39. Warning Notice 지우기: clear_warning_notice(Command).....	113
5.40. Error Notice 얻기: get_error_notice(Command).....	114
5.41. Error Notice 설정: set_error_notice(Command)	115
5.42. Error Notice 지우기: clear_error_notice(Command)	117
5.43. 동적 속성(Dynamic Property) 설정: set_property(Command)	118
5.44. 동적 속성(Dynamic Property) 불러오기 : get_property(Command).....	119
5.45. 충전: charge(Command) - Service API v1.2.6 부터 지원하지 않습니다. (CMD_DOCK으로 대체됨)	121
5.46. 주차: park(Command)	122
5.47. 주차해제: unpark(Command)	124
5.48. 가상 조이스틱 사용 시작: virtual_joystick_start(Command).....	126
5.49. 가상 조이스틱 사용 중단: virtual_joystick_finish(Command).....	127
5.50. 가상 조이스틱 헬스체크: virtual_joystick_health_check(Command)	128
5.51. 가상 조이스틱 이벤트: virtual_joystick_event(Command).....	129
6. 주요 시나리오별 워크플로우	131
6.1. 기본 초기화 과정	131
6.2. Job 수행 시 feedback event 전달 과정	132
6.3. job수행시 세부 feedback event 예시1	133
6.4. job수행시 세부 feedback event 예시2	140
7. 기타 정보	146

7.	146
7.1. worker_info와 map_info 활용 방법.....	146

1. 개요

- 본 문서는 유진로봇의 내부 개발 및 상호 비밀 유지 협약이 된 협력사를 위한 문서로 내용을 무단 복제하거나 허가하지 않은 외부로의 유출을 금지합니다.
- 본 문서는 Rocon Client Service에 대한 기본 구조를 설명하고 이 서비스를 이용하기 위해 제공되는 API의 세부 내용과 활용 방법에 관해 기술합니다.

1.1. 주요 키워드 설명

키워드	설명
Concert	Concert 는 로봇들과, 건물 내의 엘리베이터나 자동문과 같은 시설, IOT 장치, 사람들이 이용하는 스마트 기기 등이 어우러져 조화롭게 서비스를 이루는 것을 목표로 한 Robotics in Concert(ROCON)프로젝트로부터 유래되어 멀티 로봇 서비스의 설치부터 운영에 필요한 기능을 지원하는 서버 소프트웨어입니다. 멀티 로봇을 관제하기 때문에 Fleet management system(FMS)이라고 하기도 합니다.
Worker	사용자가 원하는 서비스를 수행하기 위해 필요한 기능을 보유하고 주어진 "Job"을 실제 수행하는 주체이며, 대표적으로 로봇이 있습니다.
Job	Job 은 사용자가 원하는 서비스를 수행하기 위해 제시한 목표로써 작업(Task)을 수행할 시간과 대상, 그리고 구체적인 작업 내용이 포함되는 workflow 를 기술합니다. Mission 혹은 Goal 과 유사한 개념입니다.
Rocon Client	Concert server 와 통신하는 Client 로써의 기능을 제공하는 소프트웨어를 의미합니다. 해당 로봇에게 추천되는 Job 을 수행하고 진행 상황을 알리거나 엘리베이터의 이용 등과 같이 Concert 의 도움이 필요한 상황에서 서버와 조율하여 기능을 수행하는 역할을 합니다.
Rocon Client SDK	Rocon Client 의 개발을 위해 필요한 공통 기능을 제공하는 Software Toolkit 입니다.
Rocon Client Service API	Robot service 구현을 위해 Rocon Client 를 외부에서 제어하거나 상태를 조회할 수 있는 API 형식의 정의 규약입니다. 주로 정해진 프로토콜 규격으로 요청, 응답, 피드백 등의 Message 의 규격에 대한 정의이며 이를 API 로 약칭합니다.
사용자	본문에서 언급하는 사용자는 주로 Rocon Client Service API 를 사용하여 custom application 을 개발하는 개발자를 지칭합니다. 포괄적으로 Concert Client 서비스를 사용하는 사람의 의미도 포함합니다.
Balcony	Balcony 는 사용자가 worker(로봇)에게 원하는 일을 시키기 위한 Job 을 만들거나 예약하고, 수행 중 혹은 완료된 Job 의 상태와 Job 을 수행하는 Worker 들의 상태, 수행된 작업 기록(history) 및 통계를 제공하는 사용자 인터페이스입니다. Concert 의 전체적인 상태를 조망한다는 의미에서 Balcony 로 명명되었습니다.
FMUI	FMUI(Fleet Management User Interface, 이하 FMUI)는 로봇 서비스가 운영되는 공간의 지도를 기반으로 위치, 영역에 대한 정보를 설정 및 관리하며, 로봇과 엘리베이터, 자동문 등의 상태를 모니터링할 수 있는 사용자 인터페이스입니다.

Romo_client	본문에서 언급하는 Romo 는 유진로봇에서 개발되는 Robot 의 Mobile Platform 를 SDK 에서 사용하는 약칭입니다. Robot Mobile Platform 은 로봇 자체의 핵심 구동 SW 로 로봇 기준의 서비스 주체가 되므로 이를 외부에서 핸들링 할 수 있는 서버 인터페이스(WebSocket 기반)를 가지고 있다. romo_client 는 유진 로봇의 모바일 플랫폼에 접속하는 WebSocket 클라이언트를 지칭하는 용어입니다. 로봇 모바일 플랫폼을 바라보는 클라이언트의 의미입니다.
RCM	RCM(Rocon Client Manager) 은 Rocon Client 를 관리할 수 있는 사용자 인터페이스 웹 애플리케이션입니다.

1.2. 시스템 구성

- Concert 환경
- Rocon Client Application
- 모바일 로봇

1.3. 버전 정보

- 본 문서는 다음 패키지 버전의 소프트웨어를 기준으로 작성됩니다. 소프트웨어의 업데이트에 따라 본 문서의 내용이 변경 또는 추가되며, 그에 맞춰 하단에 기입되는 패키지 버전도 수정됩니다.
 - Rocon Client : 1.0.1
 - Rocon Client Service API: 1.2.1

1.4. 개발자 지원

- 본 문서에서 언급하는 모든 API들은 실제 Robot에 설치되어 있는 RCM(Rocon Client Manager)의 개발자 모드/서비스 API테스트 페이지에서 확인 테스트가 가능합니다. 개발자 모드 확인은 RCM 로그인시 “dev”계정으로 로그인해야 확인가능합니다.

Rocon Client Service API 설명서

RCM / 설정 / RCM 설정

100% idle

YUJIN ROBOT

정보

진단

로봇 시스템

네트워크 전송

경고 고지

예외 고지

로봇 제어

기본 제어

셋업위자드

개발자

서비스 API 테스트

디버그 메시지

설정

사용자 설정

고급 설정

RCM 설정

SPT

윌리즈 노트

Rocon Client Service API 테스트

개발자를 위한 Rocon Client Service API(웹소켓 기반)의 설명과 테스트를 지원합니다.
아래의 접속과 로그인을 정상적으로 완료해야 모든 API테스트를 진행할 수 있습니다.
동시에 여러 브라우저에서 액세스한 때 하나의 연결과 로그인을 공유합니다.

Connection

Connection (CONNECTION) connected

Login / Logout

Login (CMD_LOGIN) now

API 설명

- 간략히 CMD_LOGIN으로 칭합니다.
- Rocon Client Service API 서버에 접속한 직후 디폴트 타임아웃(1분)내에 로그인 해야합니다. 로그인 되지 않으면 서버는 자동 연결 해제 합니다.
- 정상적인 로그인 이후 모든 명령과 피드백 정보를 수신할 수 있습니다.
- 정상적인 로그인시 결과에 기본 정보들이 포함되며, 이후 주기적인 피드백 정보를 받게 됩니다.

msg_body 구조

요청 메시지 열기 실행

```
{  "msg_type": "command",  "msg_body": {    "name": "login",    "params": {      "login_id": "bicho_robot",      "login_pwd": ""    }  },  "msg_uuid": "login_1",  "msg_ver": "1.0"}
```

결과 메시지 지우기

```
{  "msg_type": "result",  "msg_body": {    "result": "success"  }}
```

Websocket APIs

Connection

CONNECTION connected

Login / Logout

CMD_LOGIN now

CMD_LOGOUT

Feedback

COMMON_FEEDBACK

EVENT_FEEDBACK

General Command

CMD_CHANGE_MODE

CMD_SET_FEEDBACK

CMD_GET_RESOURCES

CMD_CREATE_JOB

CMD_CANCEL_JOB

CMD_PAUSE_JOB

CMD_RESUME_JOB

CMD_GET_PRESETS

CMD_RUN_PRESET

CMD_MOVE

CMD_STOP

CMD_PAUSE

CMD_RESUME

CMD_DOCK

CMD_DOWNLOAD_MAP_IMAGE

CMD_GET_MAP_INFO

CMD_CHANGE_MAP

CMD_RELOCATION

CMD_SET_POSITION

CMD_CLEAR_OBSACLE

CMD_GET_RECENT_FAILED_JOB

CMD_RUN_RECENT_FAILED_JOB

CMD_NOTIFY_USER_EVENT

CMD_GET_USER_EVENT

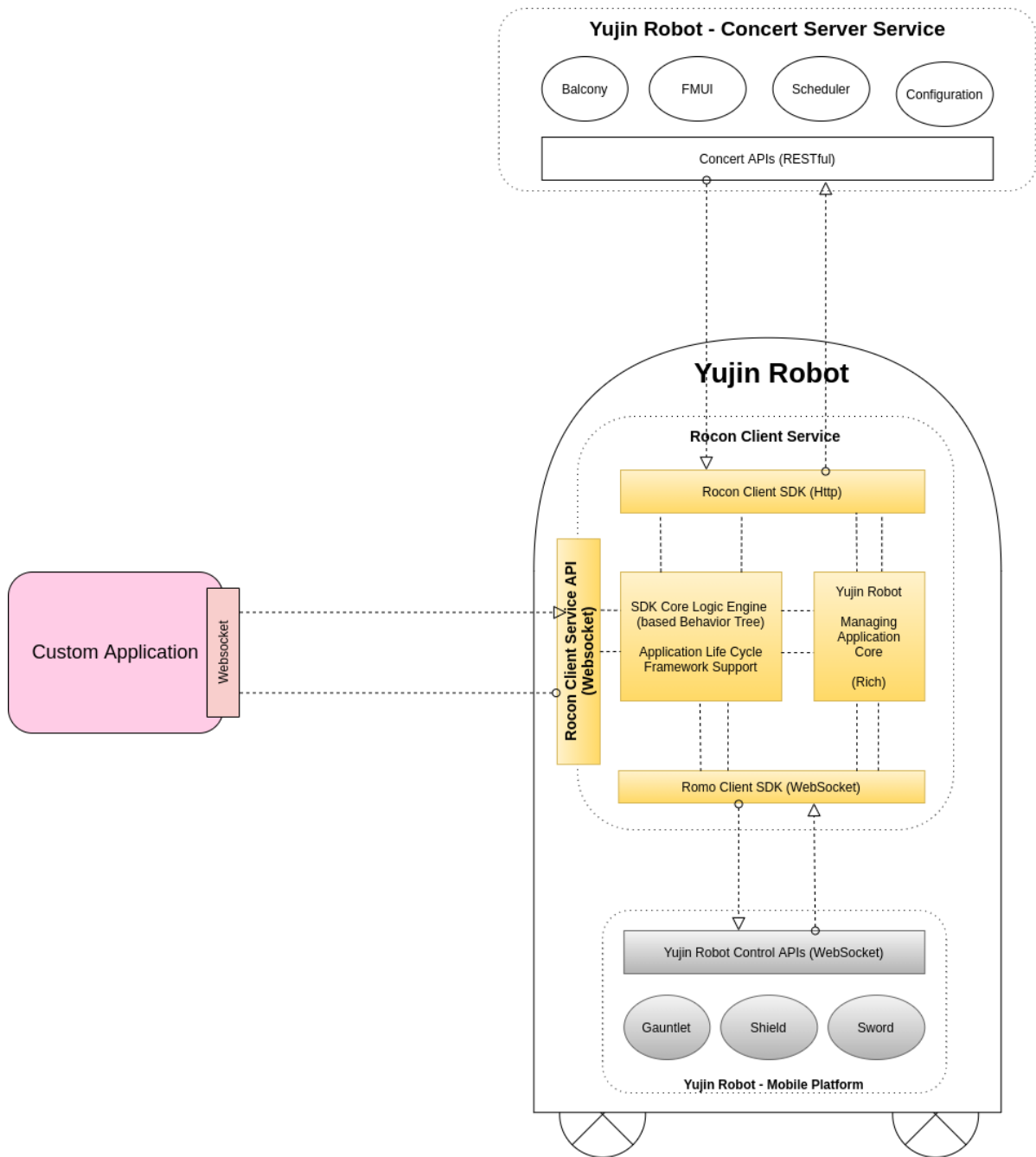
CMD_GET_GPIO

11

2. Rocon Client Service 구조와 Rocon Client Service API 설명

2.1. 개발자 지원

- Rocon Client는 Rocon Client SDK를 이용하여 작성되어 Robot의 Embedded system(Single board computer) 상에서 구동되는 백그라운드 서비스 프로그램입니다. Concert Server와 연동하는 로직 및 로봇의 공통적인 기능 수행을 돕는 Core login engine이 존재하여 주기적으로 로봇의 상태를 Concert에 업데이트하고 Concert가 로봇에 추천하는 Job이 있는지 확인하여 처리합니다. 또한 Romo Client SDK를 통해서 Yujin Robot의 Mobile Platform과 자동 연동하여 Concert에서 받은 Job을 로봇이 수행할 수 있는 세부 명령으로 변환하여 단계적 실행으로 로봇을 제어합니다. 그 밖에 다양한 외부 IOT 기기들과의 연동을 담당하는 부분들이 존재하여 로봇과 서비스에 필요한 기기들과의 연동을 가능하게 하는 핵심적인 응용 서비스 역할을 합니다. 이를 통칭해서 Rocon Client Service라 부릅니다.
- Rocon Client Service는 또한 외부의 Custom Application에서 Rocon Client Service에 접속하여 쉽게 Concert 환경에서의 로봇의 제어를 가능하게 하고 전체적인 상태를 확인할 수 있도록 돕는 API를 제공하고 있습니다. 이를 Rocon Client Service API라고 부릅니다(이하 API).



<그림 2-1 Rocon Client Service 구조>

- Mode: Rocon Client Service는 세가지('explore', 'navigate', 'service') 운영 모드가 존재합니다.
 - 'boot' - 최초로 Rocon Client가 구동되면 boot 모드로 시작합니다.
 - 'explore' - 로봇 기준으로 로봇의 이동에 필요한 지도 정보를 생성하면서 동시에 그 지도에서 현재 위치가 어디인지 지속적으로 갱신하는 로봇 운영 모드입니다. 이 모드에서는 Concert에 로봇 정보가 갱신되지 않으며 Job 할당도 이루어지지 않습니다. 로봇에 필요한 지도 정보의 생성이 필요할 때 이 모드를 통해 생성하고 Concert로 등록하기 위한 운영 모드로 주로 초기에 'service' 모드를 위한 맵 생성과 등록을 위한 모드입니다.

- 'navigate' - Concert와 통합 연동하기전에 로봇 기준으로 단위 명령을 수행할 때 사용하는 모드입니다. 이 모드에서는 Concert에서의 Job 할당 및 로봇 정보의 정보가 Concert로의 갱신이 되지 않습니다. 주로 로봇의 엔지니어링 테스트 용도로 'explore' 모드로 만든 맵에서 로봇에게 단위 테스트 명령을 직접 내려 로봇 자체의 수행을 확인할 때 사용합니다.
- 'service' - 로봇과 Concert가 통합 연동하여 Job을 수행할 수 있는 운영 모드입니다. Concert와의 유기적인 통신을 통한 Job의 수행과 모니터링을 가능하게 합니다. service 모드는 navigate 모드를 포함합니다. 최종적인 사용자 어플리케이션과 Rocon Client Service와의 정상적인 서비스는 이 모드에서 가능하게 됩니다.
- **CMD_CHANGE_MODE API를 통한 모드 변경은 'explore'와 'service'만 가능합니다.**
- Concert가 Rocon Client에 할당하는 일의 단위인 Job은 'dock', 'move', 'wait' 등 로봇을 컨트롤 하는 주요 정보와 로봇이 수행해야하는 세부 명령들의 조합으로 구성됩니다. Rocon Client는 할당 받은 Job을 해석하고 Concert와 유기적으로 통신하면서 Job의 세부 내용을 단계적으로 수행하기 위해 로봇을 제어합니다. 그리고 로봇의 세부 수행 결과나 Job의 최종 결과를 Concert로 보고 합니다. Job의 생성은 Balcony를 통해서 만들거나 Rocon Client Service SDK의 compose_job Message를 통해서 Concert에 생성 요청할 수 있습니다.
- Rocon Client Server Websocket 접속 주소
 - {로봇의 Ip address}: {포트번호}
 - 포트번호는 디폴트 10900을 사용합니다.
 - Rocon Client Service API Server의 포트번호의 변경은 RCM을 'dev' 계정으로 로그인 후 설정/고급 설정의 Rocon Client Service API Server 항목에서 변경 가능합니다.
 - 서버 포트의 변경 시 RCM의 'dev'계정으로 로그인 후 반드시 설정/RCM 설정/Rocon Client API Server의 포트도 변경해야 RCM의 정상적인 접속을 보장합니다.

3. API Message 구조

-현재 사용자 어플리케이션과(클라이언트) Rocon Client Service(서버)간에 통신 프로토콜은 Websocket 을 기반으로 합니다. 서버와 클라이언트가 주고받는 모든 API Message 는 JSON 포맷으로 구성됩니다. 모든 요청 Message 는 요청 Message 임을 표시하고 요청 내용을 정의하기 위한 필수 Message 필드를 가져야 합니다. 응답 Message 또한 Message 별로 필수 Message 필드를 가집니다.

3.1. JSON

- API에 정의된 모든 Message는 JSON 포맷으로 구성됩니다. JSON 포맷은 데이터를 텍스트 포맷으로 주고받을 수 있으며, 프로그래밍 언어나 플랫폼에 관계없이 유연하게 사용할 수 있습니다.
- JSON Message 표준에 관한 정보는 아래 사이트들의 내용을 참고하시기 바랍니다.

<https://www.JSON.org/JSON-ko.html>

<http://www.crockford.com/javascript/>

- API에서 사용하는 JSON 양식의 큰따옴표("")는 유니코드 'U+0022', HTML '"' 문자를 사용합니다. 유니코드 상의 형태가 유사한 다른 따옴표 문자를 사용할 경우 서버에서 처리하지 못할 수 있습니다. JSON 포맷은 대소문자를 구별합니다.

a. JSON Message 필드

- JSON Message 필드는 {"Name 필드": Value 필드}로 구성됩니다. Standard JSON 포맷의 Name 필드는 따옴표로 감싸진 문자열로 구성됩니다. API에서 JSON Message의 Name 필드는 **영문 소문자와 숫자, 언더바(_)** 문자만 허용합니다. Name 필드를 대문자로 입력할 경우 Message를 서버에서 처리하지 못할 수 있습니다. Value 필드는 **String, numeric(Integer, Float), Object, array, Boolean** 등을 사용할 수 있습니다.

b. Key, Value 기본 규칙

- Key, Value는 소문자로 구성합니다.
- Key의 문자열의 단어와 단어 사이는 "_"(언더바) 문자로 연결합니다.

3.2. 필수 Message 필드

- 이 절에서는 필수 Message 필드에 대해 정의하고 설명합니다. 모든 Message에 대해 공통적인 필수 Message 필드에 대해 본 챕터에서 설명하고, 명령 별 Message 세부 내용은 개별 Message 유형에 명시합니다.

필드 명	설명	자료형
msg_type	Message 의 유형을 표시합니다. command, feedback, ack(acknowledgement), result 등이 존재합니다.	String
msg_body	Message 세부 내용을 나타내는 필드입니다. Message 세부 내용에 따라서 파라미터 설정 필드 추가 및 생략될 수 있습니다.	String
		Object
msg_uuid	Message 를 구분할 수 있는 UUID 입니다. 사용자 정의 값으로 통상 msg_body 의 name 에 해당하는 스트링 + 일련번호 등으로 사용하기를 추천합니다. (사용자가 Message 를 구분하기 위한 값이므로 사용자 선택사항)	String
msg_ver	Message 규격 버전을 나타냅니다. Rocon Client Service 배포 버전입니다. 현재 고정적으로 "1.0"을 사용합니다.	String

<표 3-1 Message 의 구성 필드>

a. Message 타입: msg_type

- 해당 Message 본문이 어떤 용도로 어떤 내용을 내포할 지에 대한 정보를 대표하는 필드입니다. "msg_type" 키를 통해 표현됩니다. 해당 필드의 값으로 허용되는 문자열은 "command", "feedback", "ack", "result"입니다. 각 문자열에 대응되는 API Message의 유형은 다음 표를 참고하시기 바랍니다.

타입 문자열	API Message 유형
command	4.1 Command Message 구조
ack	4.2 Ack Message 구조
result	4.3 Result Message 구조
feedback	4.4 Feedback Message 구조

<표 3-2. API Message 유형>

b. Message 내용: msg_body

- Message 세부 내용을 나타내는 필드입니다.
- 명령의 이름인 "name" 필수 필드와 명령의 종류에 따라 세부 설정을 정의할 수 있는 "params" 필드를 가지고 있습니다. "params" 필드가 필요하지 않는 명령은 생략될 수 있습니다.

c. Message 구분자: msg_uuid

- Message를 구분할 수 있는 사용자 정의형 uuid 필드입니다. 사용자가 요청한 Message에 정의된 uui 값을 ack, result Message에 포함하여 어떤 요청 Message에 대한 응답 및 결과인지 구분하기 위한 용도로 사용합니다.

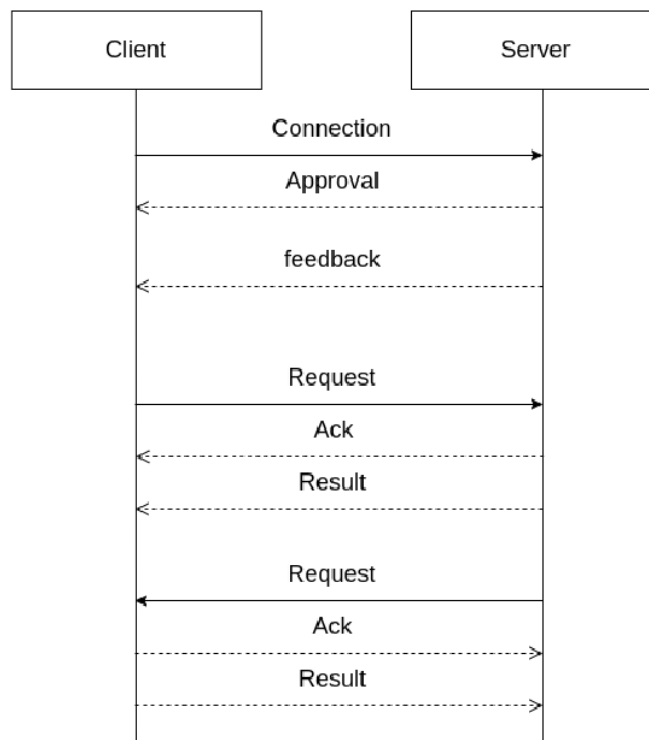
d. Message 버전: msg_ver

- Rocon Client Service API의 Message 프로토콜 버전을 기입합니다. 사용자 앱 개발 시 제공

된 Rocon Client Service API의 버전 값과 일치하는 문자열을 기입합니다. Rocon Client Service는 해당 버전 값으로 Message의 유효성을 판단하고 처리합니다. 기본적인 하위 호환성은 유지되지만 RCS의 중요 변경으로 하위 버전 호환을 유지하지 못할 경우에는 최신 Message 프로토콜 버전에 맞는 규격으로 사용자 클라이언트 앱을 수정해야 합니다.

4. API message 유형

- API를 통해 사용자의 custom service application(Client)로부터 Rocon Client Service(Server)로 전달되는 요청 Message에 관해 설명합니다. 소켓 통신이므로 경우에 따라 Rocon Client Service로부터 사용자 클라이언트에게 요청 Message를 보낼 수도 있습니다. 하지만 대부분이 클라이언트에서 서버로의 요청 Message 위주이므로 서버 측에서의 요청 Message에 설명은 추후 구체화되면 언급하도록 하겠습니다.
- 사용자 클라이언트가 Rocon Client Service에 최초 websocket 접속 요청이 서버에 전달되고 유효한 사용자로 확인될 경우 승인이 이루어지게 됩니다. 접속에 성공하는 시점부터 Rocon Client Service로부터 feedback Message를 주기적으로 받게 됩니다. feedback Message는 Rocon Client Service가 사용자 클라이언트에 제공하는 종합적인 상태 정보를 가진 Message입니다. 클라이언트가 서버에 요청하는 Request Message를 보내면 서버는 Message 수신에 대한 즉각적인 응답으로 Ack Message를 보냅니다. 그리고 해당 요청 Message를 비동기적으로 수행완료 후 Result Message를 클라이언트에 전송합니다. 이렇게 모든 Message의 기본 Handshaking 방식은 Request, Ack, Result로 순차 진행됩니다. 이러한 방식은 Server가 Client에 요청하는 Message의 경우에도 동일하게 적용됩니다.



<그림 4-1 서버-클라이언트 사이의 Message 전송>

4.1. Command Message 구조

- msg_type 필드는 "command" 키워드를 사용합니다.

- msg_body의 name 필드는 command의 종류에 따라 정해진 name을 사용합니다.
- msg_body의 params 필드는 각 command Message 별로 세부 정의된 params 형식을 사용합니다.

또한 command 의 종류에 따라 params 필드 자체가 생략될 수도 있습니다.

- msg_uuid 필드는 사용자 정의 구분용 id를 사용합니다.
- msg_ver 필드는 Rocon Client Service의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)

필드 명	설명	자료형
msg_type	"command" 현재 Message 가 "command"임을 명기합니다.	String
msg_body		Object
name	현재 command Message 의 세부 명칭입니다.	String
params	command Message 의 종류별 parameter 정보입니다. command Message 의 종류에 따라 params 필드는 생략될 수 있습니다. (5. Message 유형별 세부 설명을 참조)	Object
msg_uuid	메시지를 구분하기 위해 사용자 정의 고유 id 를 사용합니다.	String
msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

<표 4-1 Acknowledgement Message 구성 필드>

아래는 command Message 의 예시입니다.

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "command_name",
    "params": {
      # command Message 의 종류별로 params 의 구성 기입합니다.
    }
  },
  "msg_uuid": "command_name_1",
  "msg_ver": "1.0"
}
```

4.2. Ack Message 구조

- 요청 Message가 잘 전달되었는지에 대한 회신입니다.
- 수신 측의 내/외부적인 이유로 수신 받은 Message의 정상적인 처리가 불가할 때 Ack Message에 실패를 기록해 회신합니다. 이 경우 result Message는 생략됩니다.

- Ack가 실패일 경우 error_info 필드가 포함되어 error 정보가 반환됩니다.
- Ack가 성공일 경우 error_info 필드는 생략됩니다.

필드 명	설명	자료형
msg_type	"ack" 현재 Message 가 "ack"임을 명기합니다.	String
msg_body		Object
name	ack 에 해당하는 command Message 의 세부 명칭을 기입합니다.	String
result	"success" 혹은 "failure"로 반환됩니다. "success"는 정상 수신되었으며 내부 처리 가능할 경우 회신 되며 추후 result Message 를 다시 회신합니다. "failure"는 정상 수신되었으나 내부 처리가 불가할 경우 회신 되며 result Message 는 생략됩니다.	String
error_info	result 가 failure 일때 error_info 필드가 추가됩니다. result 가 success 일때 error_info 필드는 생략됩니다.	Object
code	Error code 를 기입합니다.	String
description	Error code 에 해당되는 기본 설명이 포함됩니다.	String
msg_uuid	메시지를 구분하기 위해 사용자 정의 고유 id 를 사용합니다.	String
msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

<표 4-2 Acknowledgement Message 구성 필드>

아래는 ack Message 의 예시입니다. (success 일 경우)

```
{
  "msg_type": "ack",
  "msg_body": {
    "name": "commmand_name",
    "result": "success"
  },
  "msg_uuid": "command_name_1",
  "msg_ver": "1.0"
}
```

아래는 ack Message 의 예시입니다. (failure 일 경우)

```
{
  "msg_type": "ack",
  "msg_body": {
    "name": "commmand_name",
    "result": "failure",
    "error_info": {
```

```

    "code": "1024",
    "description": "Example command description"
  }
},
"msg_uuid": "command_name_1",
"msg_ver": "1.0"
}

```

4.3. Result Message 구조

- 요청 Message에 대한 최종 결과에 대한 회신입니다.
- 요청 Message 처리의 실패일 경우 error_info 필드가 포함되어 error 정보가 반환됩니다.
- 요청 Message 처리의 성공일 경우 error_info 필드는 생략됩니다.
- 요청 Message가 Data를 포함할 경우 data_contents 필드가 포함되어 반환됩니다.
- 요청 Message가 Data를 포함하지 않는 단순 요청의 경우 data_contents 필드가 포함되지 않습니다.

필드 명	설명	자료형
msg_type	"result" 현재 Message 가 "result"임을 명기합니다.	String
msg_body		Object
name	result 에 해당하는 command Message 의 세부 명칭을 기입합니다.	String
result	"success" 혹은 "failure"로 반환됩니다. "success"는 정상 처리되었음을 나타냅니다. "failure"는 실패했음을 나타내며 error_info 필드가 추가되어 회신 됩니다.	String
error_info	result 가 failure 일때 error_info 필드가 추가됩니다. result 가 success 일때 error_info 필드는 생략됩니다.	Object
code	Error code 가 기입됩니다.	String
description	Error code 에 해당되는 기본 설명이 포함됩니다.	String
data_contents_type	반환되는 data_contents 의 타입이 기록됩니다. "maps", "waypoints", "charging-stations" 등의 타입이 기록됩니다.	String
data_contents	data 반환을 요청하는 Message 의 경우 data_contents 필드에 해당 정보가 추가됩니다.	Object
msg_uuid	사용자 정의 구분용 id 를 사용합니다.	String
msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

<표 4-3 Result Message 구성 필드>

아래는 result Message 의 예시입니다. (success 일 경우, 단순 command 요청일 경우)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "commmand_name",
    "result": "success"
  },
  "msg_uuid": "command_name_1",
  "msg_ver": "1.0"
}
```

아래는 result Message 의 예시입니다. (success 일 경우, data 반환을 요청하는 command 일 경우)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "commmand_name",
    "result": "success",
    "data_contents": {
      # 해당 command 의 요청 data 가 기입됩니다 (command 별 세부 내용 참조)
    }
  },
  "msg_uuid": "command_name_1",
  "msg_ver": "1.0"
}
```

아래는 result Message 의 예시입니다. (failure 일 경우)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "commmand_name",
    "result": "failure",
    "error_info": {
      "code": "2058",
      "description": "Error occurred: move a to b"
    }
  },
  "msg_uuid": "command_name_1",
  "msg_ver": "1.0"
}
```

4.4. Feedback Message 구조

- Feedback Message는 주로 서버에서 클라이언트로 종합적인 상태 정보를 주기적으로 전달할 때 발생합니다. (msg_body의 name이 "common"인 경우)
- Feedback Message는 서버에서의 중요한 동적인 변경사항이 발생했을 때 해당 이벤트의 정보를 포함하여 클라이언트에 전달됩니다. (msg_body의 name이 "event_xxx"인 경우)
- msg_body의 내용은 정보성 데이터로 구성되며 각 모드별로 내용이 다를 수 있습니다.
- msg_body의 name은 주기적으로 worker의 기본 정보를 알려주는 용도의 "common"과 특정 이벤트의 발생에 대한 정보를 보내는 주는 "event_xxx"로 구성(xxx는 임의의 명칭)됩니다.
- Event_xxx는 다음과 같은 종류가 있습니다.

msg_body 의 name	설명
"common"	로봇의 상태정보를 주기적으로 client 에게 통보됩니다.
"event_map_changed"	로봇이 job 을 수행 중에 텔레포터(엘리베이터)를 이용해 동적으로 현재의 맵을 변경했을 때 변경에 대한 feedback event 로 client 에게 통보됩니다.
"event_task_changed"	Concert 에 등록된 로봇 task 의 구성이 변경되었을 때 feedback event 로 client 에 통보됩니다. FMUI 상에서 새로운 리소스를 추가하거나 수정 후 task 에 적용하기 위해 task_definition 에서 해당 task 를 삭제했을 때 worker 는 새로운 task 를 자동으로 만들어 등록하고 이 때 feedback event 가 client 에게 통보됩니다. client 는 현재 맵의 리소스를 갱신하기 위해 "download_map_image" 커맨드를 호출해 이후 맵의 정보를 업데이트해야 합니다.
"event_job_progress"	새로운 job 을 생성했을 때 생성된 job 의 주요 진행 사항을 feedback event 로 client 에게 통보합니다. 주로 started, finished, paused, resumed, failed 등의 주요 상태 변경 시 통보됩니다.
"event_status_changed"	Rocon Client 의 상태 변화가 생길 때 명시적으로 feedback event 로 client 에게 통보됩니다. Common Feedback 의 worker_info.status 값으로 변경이 발생했을 때 발생합니다.
"event_button"	로봇의 물리 버튼의 상태 변화가 생길 때 명시적으로 feedback event 로 client 에게 통보됩니다. 통보되는 물리 버튼의 명칭은 로봇마다 다를 수 있습니다. Gocart180, MSCMK, CSSD 로봇의 경우 "button_start_switch", "button_x"가 포함됩니다. Gocart60,120 의 경우 "button_o", "button_x"가 포함됩니다.

"event_mode_changed"	change_mode 로 모드가 변경되면 명시적으로 feedback event 로 client 에게 통보됩니다. 'service', 'navigate', 'explore' 3 가지 모드 변경이 이루어질 때 통보됩니다.
"event_robot_new_map_saved"	explore 모드에서 사용자가 맵을 그리고 CMD_SAVE_ROBOT_NEW_MAP 으로 저장했을 때 명시적으로 client 에게 통보됩니다. (해당 기능은 현재 개발 중입니다.)
"event_robot_new_map_explore"	explore 모드에서 사용자가 joystick 으로 맵을 그리는 동안 발생하는 feedback 이벤트이며 현재 생성중인 맵의 윤곽을 확인할 수 있는 데이터 정보를 포함해서 통보됩니다. (해당 기능은 현재 개발 중입니다.)

아래 규격은 "service"모드에서의 "common" 피드백 정보입니다.

필드 명	설명	자료형
msg_type	"feedback" 현재 Message 가 "feedback"임을 명기합니다.	String
msg_body		Object
name	feedback 에 해당하는 명칭이 기입됩니다. 현재 feedback 은 공통 정보에 해당하는 "common"이 명기됩니다.	String
mode	Rocon Client Service 에 현재 설정된 모드를 나타냅니다.	String
worker_info		Object
type	Worker 의 유형을 지칭합니다. 현재 로봇인 경우 "robot"으로 고정됩니다.	String
name	Worker 를 명칭하는 고유 이름이 기록됩니다.	String
uuid	Worker 의 고유 uuid 가 기록됩니다.	String
task_name	Worker 가 사용하는 task 이름이 기록됩니다.	String
task_description	Worker 가 사용하는 task 의 설명이 기록됩니다.	String
status	Worker 의 현재 상태가 기록됩니다.	String
status_toggle	Worker 의 현재 복합 상태가 기록됩니다. 문자열 배열 구조	Array
status_p	Worker 의 현재 최우선 상태가 기록됩니다.	String
robot_width	로봇의 폭 정보가 기록됩니다	Float
robot_length	로봇의 길이 정보가 기록됩니다.	Float
robot_navi_mode	로봇의 navi_mode 정보가 기록됩니다.	String
map	Worker 가 위치하고 있는 지도 정보 필드입니다.	Object
id	map 의 고유 id 입니다. (Concert 에서 지정된 고유 id 값입니다)	String
name	map 에 부여된 이름입니다.	String
pose	Worker 의 현재 위치 정보 필드입니다.	Object
x	맵에서의 Worker 의 좌표 x 정보입니다.	Float

	y	맵에서의 Worker 의 좌표 y 정보입니다.	Float
	theta	맵에서의 Worker 의 좌표 방향 정보입니다. (radian 값)	Float
	velo	로봇 진행 시 상대 속도를 나타냅니다. 이 정보는 rocon_client_service 의 내부 설정에 따라 feedback 에 포함됩니다.	Object
	velo_dx	전/후진 방향 속도(m/s)	Float
	velo_dy	좌/우 방향 속도(m/s, mecanum)	Float
	velo_dz	회전 방향 속도(rad/s)	Float
	battery	Worker 의 현재 배터리 정보 필드입니다.	Object
	battery_level	현재 배터리의 사용 용량을 퍼센트로 표시합니다.	Integer
	now_charging	true / false true 의 경우 현재 charging 중임을 나타냅니다. (통상 docking station 에 접속해 있거나 전원 케이블을 통한 충전 중인 경우입니다. false 의 경우 배터리로 구동중인 상태입니다. (명세로봇의 경우, 이 필드는 사용하지 않습니다(항상 false). 충전중인 상태 확인은 charge_source 가 "dcjack" 혹은 "none"으로 판단하면 됩니다.)	Boolean
	charge_source	충전시 공급되는 전원의 타입을 나타냅니다. 현재 3 가지로 정의됩니다. "docker": charging station "dcjack": dc power "none": no charging source	String
	brake_released	true / false true 의 경우 로봇의 브레이크가 해제된 상태로 수동으로 로봇을 움직일 수 있는 상태입니다. 이 경우 자동 명령을 수행할 수 없습니다. false 의 경우가 일반적으로 자동 명령을 수행할 수 있는 기본 상태입니다.	Boolean
	manual_mode	로봇의 manual_mode 버튼으로 전환된 경우 true, 자율 주행모드로 전환된 경우 false 를 기록합니다.	Boolean
	emergency_alarm	로봇이 비상 상황의 알람음을 감지했을 때 true, normal 상태일때 false 를 기록합니다. (비상 상황 알람음은 해외 요구사항 등 특정 내부 설정에 의해 활성화됩니다)	Boolean
	emergency_button	로봇의 emergency 버튼을 눌러 on 상태일때 true, off 상태일때 false 를 기록합니다.	Boolean
	paused	true / false true 의 경우 로봇이 일시 멈춤 상태입니다. 이 경우는 로봇에 부착된 pause 버튼이 토근 된 경우입니다. 일시 멈춤이 해제되면 false 상태가 됩니다.	Boolean
	error_info	Worker status 가 "error"인 경우 error_info 필드가 추가됩니다.	Object

		이 error 정보는 worker 의 종합적인 상태가 정상 수행을 할 수 없는 error 상태로 이에 해당되는 세부 정보가 포함됩니다.	
	code	Error code 를 기입합니다.	String
	description	Error code 에 해당되는 기본 설명이 포함됩니다.	String
	job_id	현재 실행 중인 job 의 id 입니다. status 가 'busy'상태 일 때 만 키가 추가됩니다.	String
	path_plan	로봇의 주행 경로 계획 정보가 기록됩니다. 이 정보는 rocon_client_service 의 내부 설정에 따라 feedback 에 포함됩니다. (Default 는 포함되지 않습니다, 일반 Custom Client 구현에는 참조하지 않아도 됩니다)	Object
	odometry	로봇의 주행과 관련된 속도 정보가 기록됩니다. 이 정보는 rocon_client_service 의 내부 설정에 따라 feedback 에 포함됩니다. (Default 는 포함되지 않습니다, 일반 Custom Client 구현에는 참조하지 않아도 됩니다)	Object
	virtual_worker	Virtual worker 인지 판별하는 값으로 virtual worker 라면 True, 실제 로봇일 경우 False 의 값을 가집니다. (Virtual worker 의 경우 해당 필드가 추가됩니다)	Boolean
	concert_info	Concert 의 상태 정보가 기록됩니다.	Object
	scheduler_url	Rocon Client 에 설정된 scheduler 의 접속 URL 을 표시합니다.	Object
	iot_operator_url	Rocon Client 에 설정된 iot_operator 의 접속 URL 을 표시합니다.	String
	site_config_url	Rocon Client 에 설정된 site_config 의 접속 URL 을 표시합니다.	String
	worker_config_url	Rocon Client 에 설정된 worker_config 의 접속 URL 을 표시합니다.	String
	scheduler	Concert 의 핵심인 Scheduler 의 상태 정보를 표시합니다.	Object
	ver	Scheduler 버전 정보를 표시합니다.	String
	status	주기적으로 Scheduler 와 접속이 정상적인지 표시합니다. 정상적인 접속 확인되면 "connected"로, 그렇지 않으면 null 값으로 표시됩니다.	String
	checked_datetime	마지막 확인된 날짜시간 정보를 기록합니다. (UTC 기준)	String
	msg_uuid	사용자 정의 구분용 id 를 사용합니다. feedback 의 경우 date_time 이 기록됩니다. (UTC 기준)	String
	msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

<표 4-4 Feedback Message 구성 필드>

Feedback Message 예시(common)

```
{
  "msg_type": "feedback",
  "msg_body": {
    "name": "common",
    "mode": "service",
```

```

"worker_info": {
  "type": "robot",
  "name": "vw_custom_g_blcho",
  "uuid": "vw_custom_g_blcho",
  "task_name": "blcho Test",
  "task_description": "blcho Test",
  "status": "busy",
  "status_toggle": [],
  "status_p": "busy ",
  "robot_width": 0.51,
  "robot_length": 0.73,
  "map": {
    "id": "5f4f2de9429b4a0012d40c54",
    "name": "3F"
  },
  "pose": {
    "x": 1.09000000000000034,
    "y": 0.3805650684931514,
    "theta": 0.0
  },
  "velo": null,
  "battery": {
    "battery_level": 84.800000000000005,
    "now_charging": false,
    "charge_source": "docker"
  },
  "brake_released": false,
  "paused": false,
  "error_info": null,
  "job_id": "5ffe4b51e6a14b002aa29e12",
},
"concert_info": {
  "scheduler_url": "192.168.10.246:10602/scheduler",
  "iot_operator_url": "192.168.10.246:10702",
  "site_config_url": "192.168.10.246:10602/configuration",
  "worker_config_url": "192.168.10.246:10604",
  "scheduler": {
    "ver": "1.8.2",
    "status": "connected",
    "checked_datetime": "2021-07-01T06:37:52.56Z"
  }
}

```

```

    }
  },
  "msg_uuid": "feedback",
  "msg_ver": "1.0"
}

```

아래 규격은 "service"모드에서의 "event_map_changed" 피드백 정보입니다.

필드 명	설명	자료형
msg_type	"feedback" 현재 Message 가 "feedback"임을 명기합니다.	String
msg_body		Object
name	feedback 에 해당하는 명칭이 기입됩니다. "event_map_changed"가 명기됩니다.	String
mode	Rocon Client Service 에 현재 설정된 모드를 나타냅니다.	String
worker_info	worker 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
concert_info	Concert 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
event_info	event_map_changed 의 내용이 포함됩니다.	Object
result	"success", "failure"의 결과가 기록됩니다. (Client UI 에서 map 정보를 활용해 화면을 구성한다면 "success" 확인 이후 맵 다운로드: download_map_image(Command)를 호출하여 맵 정보 및 맵 이미지를 다운로드하여 화면을 업데이트하면 됩니다.)	String
error_info	result 가 "failure"인 경우 error_info 필드가 추가됩니다. ("code", "description"이 포함된 공통 error_info 필드)	Object
msg_uuid	사용자 정의 구분용 id 를 사용합니다. feedback 의 경우 date_time 이 기록됩니다.	String
msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

Feedback Message 예시(event_map_changed)

```

{
  "msg_type": "feedback",
  "msg_body": {
    "name": "event_map_changed",
    "mode": "service",
    "worker_info": {
      # worker_info 일반 사항 포함됩니다.
    },
  },
}

```

```

    "event_info": {
        "result": "success"
    },
    "msg_uuid": "feedback",
    "msg_ver": "1.0"
}

```

아래 규격은 "service"모드에서의 "event_task_changed" 피드백 정보입니다.

필드 명	설명	자료형
msg_type	"feedback" 현재 Message 가 "feedback"임을 명기합니다.	String
msg_body		Object
name	feedback 에 해당하는 명칭이 기입됩니다. "event_task_changed"가 명기됩니다.	String
mode	Rocon Client Service 에 현재 설정된 모드를 나타냅니다.	String
worker_info	worker 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
concert_info	Concert 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
event_info	event_task_changed 의 내용이 포함됩니다.	Object
result	"success", "failure"의 결과가 기록됩니다. (Task 변경의 경우 FMUI 의 리소스 변경시에 이루어 지므로 Client UI 에서 map 정보를 활용해 화면을 구성한다면 "success" 확인 이후 맵 다운로드: download_map_image(Command)를 호출하여 맵 정보 및 맵 이미지를 다운로드하여 화면을 업데이트하면 됩니다.)	String
error_info	result 가 "failure"인 경우 error_info 필드가 추가됩니다. ("code", "description"이 포함된 공통 error_info 필드)	Object
msg_uuid	사용자 정의 구분용 id 를 사용합니다. feedback 의 경우 date_time 이 기록됩니다.	String
msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

Feedback Message 예시(event_task_changed)

```

{
    "msg_type": "feedback",
    "msg_body": {
        "name": "event_task_changed",
        "mode": "service",

```

```

"worker_info": {
    # worker_info 일반 사항 포함됩니다.
},
"event_info": {
    "result": "success"
}
},
"msg_uuid": "feedback",
"msg_ver": "1.0"
}
    
```

아래 규격은 "service"모드에서의 "event_job_progress" 피드백 정보입니다.

필드 명	설명	자료형
msg_type	"feedback" 현재 Message 가 "feedback"임을 명기합니다.	String
msg_body		Object
name	feedback 에 해당하는 명칭이 기입됩니다. "event_job_progress"가 명기됩니다.	String
mode	Rocon Client Service 에 현재 설정된 모드를 나타냅니다.	String
worker_info	worker 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
concert_info	Concert 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
event_info	event_job_progress 의 내용이 포함됩니다.	Object
result	"success", "failure"의 결과가 기록됩니다.	String
data		Object
job_id	현재 수행중인 Job 의 id 가 기록됩니다.	String
busy_status	event_job_progress 는 worker 가 busy 상태로 변경된 직후와 "idle"상태로 전환되기 직전까지의 busy 상태의 세부 상태를 파악하기 위한 이벤트입니다. 통상 "started", "finished" 쌍 혹은 "started", "cancelled" 또는 "aborted"의 쌍으로 이벤트가 발생합니다. busy_status 의 아래와 같은 세부 정의를 가지고 있습니다. "started": job 수행 시작할 때 "finished": job 수행 종료할 때 "cancelled": balcony 통한 Job 취소할 때 "aborted": 로봇 내부의 예외 상황으로 job 을 강제 abort 시킬 때	String

		"soft_paused": 주행 중 SW 적인 조작으로 일시 멈춤 상태일때 "soft_resumed": soft_paused 가 해제되어 다시 움직일 때 추가로 세부 action 들의 stated, finished 관련 정의함. 가령, move, dock, lock, unlock, precision_park, use_resource, wait 등 세부 action 을 수행 중이라면 busy_status 가 {action 명}_started, {action 명}_running, {action 명}_finished 로 된 event_job_progress 가 전송됩니다. {action 명}_running 의 경우 해당 action 이 내부적으로 복합적인 세부 action 들을 수행하는 경우에 따라 발생합니다. 만약 현재 위치에서 특정 waypoint 로의 이동 시작과 종료에 대한 이벤트를 확인하고 싶은 경우 "move_started"가 발생되며 이동 완료 후에 "move_finished"가 발생합니다. 이러한 세부 event 의 내용을 확인할 수 있는 target_info 필드가 추가되며 각 세부 action 의 내용을 담고 있습니다. 하단의 job 수행시 세부 feedback event 예시 1, 예시 2 를 참고하세요.	
	busy_status_reason	busy_status 의 원인에 대한 키워드 설명이 포함됩니다.	String
	target_info		Object
	action_name	각 세부 action 의 명칭이 기록됩니다. ('move', 'dock', 'lock', 'unlock', 'precision_park', 'use_resource', 'wait' 등)	String
		추가되는 내용은 각 action 별로 상이합니다. 하단의 action 별 target_info 항목을 참고하세요.	
	error_info	result 가 "failure"인 경우 error_info 필드가 추가됩니다. ("code", "description"이 포함된 공통 error_info 필드)	Object
msg_uuid		사용자 정의 구분용 id 를 사용합니다. feedback 의 경우 date_time 이 기록됩니다.	String
msg_ver		Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

action 별 target_info

Action	필드 명	설명	자료형
move	action_name	'move'가 기록됩니다.	String
move	waypoint_name	FMS 에서 만든 waypoint 의 alias 명칭이 기록됩니다. Rocon_client 내부 자동 계산된 지점의 경우 아래와 같은 명칭이 기록됩니다. 'auto_move_entry_pose': 자동문이나 엘리베이터 사용을 위해 자동 계산된 지점까지 이동시에 기록됩니다. 'auto_move_exit_pose': 자동문이나 엘리베이터 사용 종료 지점까지 이동시에 기록됩니다. 'auto_dock_entry_pose': docking 을 위해 marker 전방 자동 계산된 임의의 위치까지 이동시에 기록됩니다.	String
	waypoint_id	지정된 waypoint 의 id 가 기록됩니다. 임의의 내부 계산된 pose 일 경우 null 이 기록될 수도 있습니다.	
	finishing_timeout	설정된 타임아웃(minute 기준) 값이 기록됩니다. 디폴트 -1 은 무한.	Integer
dock	action_name	'dock'이 기록됩니다.	String
	station_name	해당 docking station 의 alias 명칭이 기록됩니다.	String
	marker_value	해당 docking station 의 marker id 가 기록됩니다.	String
	finishing_timeout	설정된 타임아웃(minute 기준) 값이 기록됩니다. 디폴트 -1 은 무한.	Integer
lock	action_name	'lock'이 기록됩니다.	String
	resource_name	lock 을 사용하는 리소스의 alias 명칭이 기록됩니다.	String
unlock	action_name	'unlock'이 기록됩니다.	String
	resource_name	unlock 을 사용하는 리소스의 alias 명칭이 기록됩니다.	String
precision_park	action_name	'precision_park'이 기록됩니다.	String
	marker_value	Marker id 값이 기록됩니다.	String
	dist_meter_from_marker	정밀 주차할 Marker 앞 지정된 거리 값이 기록됩니다. (미터 단위)	Float
	finishing_timeout	설정된 타임아웃(minute 기준) 값이 기록됩니다. 디폴트 -1 은 무한.	Integer
use_resource	action_name	'user_resource'가 기록됩니다. 이 action 은 autodoor, elevator 등을 이용할 때 자동 생성됩니다.	String

	resource_type	리소스 타입이 기록됩니다. 'Autodoor', 'Teleporter', 'ExclusiveArea' 등이 기록됩니다. (Teleporter 는 엘리베이터를 지칭하는 내부 리소스 명칭입니다)	String
wait	action_name	'wait'가 기록됩니다.	String
	wait_type	지정된 wait type 이 기록됩니다. 'duration', 'signal', 'human_input'	String
	wait_timeout	지정된 wait action 의 timeout 값이 기록됩니다. (second 기준)	integer
boarding	action_name	'boarding'이 기록됩니다. 이 action 은 elevator 를 탑승할 때 자동 생성됩니다.	String
	resource_name	'Teleporter'가 기록됩니다	String
unboarding	action_name	'unboarding'이 기록됩니다. 이 action 은 elevator 에서 내릴 때 자동 생성됩니다.	String
	resource_name	'Teleporter'가 기록됩니다	String
elevator_call	action_name	'elevator_call'이 기록됩니다. 이 action 은 elevator 호출시에 자동 생성됩니다.	String

Feedback Message 예시(event_job_progress - finished)

```
{
  "msg_type": "feedback",
  "msg_body": {
    "name": " event_job_progress ",
    "mode": "service",
    "worker_info": {
      # worker_info 일반 사항 포함됩니다.
    },
    "event_info": {
      "result": "success",
      "data": {
        "job_id": "607688fa1f416d002bedccc4",
        "busy_status": "finished",
        "busy_status_reason": ""
      }
    }
  },
  "msg_uuid": "2021-04-14T06:17:30.538Z",
}
```

```
"msg_ver": "1.0"
}
```

아래 규격은 "service"모드에서의 "event_status_changed" 피드백 정보입니다.

event_status_changed 는 변경된 status 혹은 status_reason 이 존재할 때 발생합니다. 즉 같은 "idle" status 이더라도 "status_p_reason"이 다른 내용으로도 event 가 발생할 수 있습니다.

필드 명	설명	자료형
msg_type	"feedback" 현재 Message 가 "feedback"임을 명기합니다.	String
msg_body		Object
name	feedback 에 해당하는 명칭이 기입됩니다. "event_status_changed"가 명기됩니다.	String
mode	Rocon Client Service 에 현재 설정된 모드를 나타냅니다.	String
worker_info	worker 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
concert_info	Concert 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
event_info	event_status_changed 의 내용이 포함됩니다.	Object
result	"success", "failure"의 결과가 기록됩니다.	String
data		Object
prev_status_p	변경되기 전 status_p 가 기록됩니다.	String
prev_status_p_reason	변경되기 전 status_p 의 reason 이 기록됩니다 (prev_status_p_reason 은 키워드를 통해 상황 판단을 이해하기 위한 참고용 정보입니다)	String
status_p	현재 변경된 status_p 가 기록됩니다.	String
status_p_reason	현재 변경된 status_p 의 reason 이 기록됩니다 (status_p_reason 은 키워드를 통해 상황 판단을 이해하기 위한 참고용 정보입니다)	String
error_info	result 가 "failure"인 경우 error_info 필드가 추가됩니다. ("code", "description"이 포함된 공통 error_info 필드)	Object
msg_uuid	사용자 정의 구분용 id 를 사용합니다. feedback 의 경우 date_time 이 기록됩니다.	String
msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

Feedback Message 예시(event_status_changed)

```
{
```

```

"msg_type": "feedback",
"msg_body": {
  "name": "event_status_changed",
  "mode": "service",
  "worker_info": {
    # worker_info 일반 사항 포함됩니다.
  },
  "event_info": {
    "result": "success",
    "data": {
      "prev_status_p": "busy",
      "prev_status_p_reason": "busy_start_working",
      "status_p": "idle",
      "status_p_reason": "idle_done_job_successfully"
    }
  }
},
"msg_uuid": "2021-04-14T00:56:04.692Z",
"msg_ver": "1.0"
}

```

아래 규격은 "service"모드에서의 "event_button" 피드백 정보입니다.

필드 명	설명	자료형
msg_type	"feedback" 현재 Message 가 "feedback"임을 명기합니다.	String
msg_body		Object
name	feedback 에 해당하는 명칭이 기입됩니다. "event_button"가 명기됩니다.	String
mode	Rocon Client Service 에 현재 설정된 모드를 나타냅니다.	String
worker_info	worker 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
concert_info	Concert 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
event_info	event_button 의 내용이 포함됩니다.	Object
result	"success", "failure"의 결과가 기록됩니다.	String
data		Object
button_name	버튼을 정의하는 명칭이 기록됩니다. Gocart180, MSCMK 등 모델에 적용된 버튼 중 event 로 확인할 수 있는	String

		명칭은 다음과 같습니다. "button_start_switch": Start 버튼입니다. "button_x": Cancel 버튼입니다. Gocart60,120 의 O, X 버튼은 각각 "button_o", "button_x"로 확인됩니다.	
	button_state	버튼의 상태를 정의한 값입니다. 1: 버튼을 누를 때(pressed) 발생하는 state 값입니다. 3: 버튼을 땔 때(released)발생하는 state 값입니다.	Integer
	error_info	result 가 "failure"인 경우 error_info 필드가 추가됩니다. ("code", "description"이 포함된 공통 error_info 필드)	Object
	msg_uuid	사용자 정의 구분용 id 를 사용합니다. feedback 의 경우 date_time 이 기록됩니다.	String
	msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

Feedback Message 예시(event_button)

```
{
  "msg_type": "feedback",
  "msg_body": {
    "name": "event_button",
    "mode": "service",
    "worker_info": {
      # worker_info 일반 사항 포함됩니다.
    },
    "event_info": {
      "result": "success",
      "data": {
        "button_name": "button_start_switch",
        "button_state": 3
      }
    }
  },
  "msg_uuid": "2021-04-14T01:36:58.863Z",
  "msg_ver": "1.0"
}
```

아래 규격은 "event_mode_changed" 피드백 정보입니다.

필드 명	설명	자료형
------	----	-----

msg_type	"feedback" 현재 Message 가 "feedback"임을 명기합니다.	String
msg_body		Object
name	feedback 에 해당하는 명칭이 기입됩니다. "event_mode_changed"가 명기됩니다.	String
mode	Rocon Client Service 에 현재 설정된 모드를 나타냅니다.	String
worker_info	worker 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
concert_info	Concert 의 상태 정보가 기록됩니다. 위의 "common" feedback 에 포함되는 worker_info 와 동일한 구조로 포함됩니다.	Object
event_info	event_button 의 내용이 포함됩니다.	Object
result	"success", "failure"의 결과가 기록됩니다.	String
data		Object
mode	변경된 mode 정보가 기록됩니다. (‘service’, ‘navigate’, ‘explore’ 중 하나)	String
error_info	result 가 "failure"인 경우 error_info 필드가 추가됩니다. ("code", "description"이 포함된 공통 error_info 필드)	Object
msg_uuid	사용자 정의 구분용 id 를 사용합니다. feedback 의 경우 date_time 이 기록됩니다.	String
msg_ver	Rocon Client Service 의 버전 명칭을 사용합니다. (현재 고정적으로 1.0)	String

Feedback Message 예시(event_mode_changed)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "change_mode",
    "result": "success",
    "data_contents": {
      "current_mode": "explore"
    }
  },
  "msg_uuid": "change_mode",
  "msg_ver": "1.0"
}
```

4.5. Worker의 Status 세부 설명

Rocon_client 에서 사용하는 status 용어는 Workrer(Robot)의 현재 상태에 대한 정의입니다. V1.2.1 부터 기존 status 개념이 status, status_p, status_toggle 로 확장되었습니다. 확장된 status 개념은 Concert 시스템

상에서 이 상태는 로봇(클라이언트)과 Concert(서버)간에 서로 완전히 일치된 상태 값을 공유하며 각각의 편에서 그 상태에 맞는 행동을 수행하게 되어 있습니다. worker 의 status 정보는 기본적으로 feedback 메시지에 포함되는 worker_info 의 status 필드에 해당 상태를 나타내는 문자열로 표시됩니다.

4.5.1. status 정의

- init, idle, busy, explore, idle_on_hold, busy_on_hold, suspend, idle_auto_charging, offline 중 하나로 사용합니다.

	Status	Description
1	"init"	Rocon_client 가 동작하는 최초의 상태 정의입니다. 각종 초기화 과정을 수행합니다.
2	"idle"	Rocon_client 가 초기화를 끝내고 정상적인 job 을 수행할 수 있는 대기 상태의 정의 입니다. 또한 "busy" 상태에서 job 수행 완료하거나 취소, 강제 취소등에 의해 다시 대기 상태로 돌아가는 경우의 상태이기도 합니다. "idle"상태에서 항상 Concert 에서 추천하는 Job 을 새로 할당 받을 수 있습니다.
3	"busy"	"idle"상태에서 Concert 로부터 새로운 job 을 할당 받고 해당 job 을 실행하는 상태의 정의입니다.
4	"explore"	Rocon_client 의 mode 가 "explore"로 지정되면 설정되는 상태입니다. 즉, 로봇 역시 맵 생성을 위한 explore 상태이므로 사용자는 조이스틱을 이용해 맵 생성을 위한 탐색을 할 수 있습니다.
5	"idle_on_hold"	Rocon_client 가 현재 수행하는 job 이 없는 idle 상태이지만 내부 조건에 의해 Concert 로 부터 job 을 할당 받지 못하는 경우입니다. 예를 들어, idle 상태에서 로봇의 brake 를 released 했다면 status 는 idle_on_hold 이며, status_p 는 brake_released 상태, status_toggle 은 ['break_released']로 됩니다. 이 경우 Idle_on_hold 상태이므로 Concert 에서 작업을 할당 받지 않습니다.
6	"busy_on_hold"	Rocon_client 가 현재 job 을 수행중에 내부 조건에 의해 일시적으로 진행되지 못하는 경우입니다. 예를 들어, busy 상태에서 로봇이 움직이고 있는 도중 brake 를 released 하고 emergency_button 을 눌렀다면 status 는 busy_on_hold 이며, status_p 는 우선순위가 가장 높은 emergency_button 상태, status_toggle 은 ["break_released", "emergency_button"]로 됩니다.
7	"suspend"	RCM 의 설정에서 Suspend 기능이 enabled 될 때 이용 가능한 상태입니다.

		현재 진행 중인 Job 을 사용자가 취소하거나 로봇 자체의 실패로 중단될 때 로봇은 'suspend'상태로 됩니다. 이 경우 사용자가 RCM 의 설정 메뉴를 통해 직접 resume 하거나 CMD_RESUME_FOR_SUSPEND API 를 통해서 resume 처리를 할 수 있습니다.
8	"idle_auto_charging"	Rocon_client 의 설정에서 idle_auto_charging 기능이 enabled 할때 이용 가능한 상태입니다. Idle_auto_charging 기능은 로봇의 충전 상태를 percentage 기준으로 자동 충전 시작과 충전 완료로 일을 시작할 수 있는 기준을 미리 설정해 두고 해당 조건이 맞으면 자동 충전을 진행하거나 자동 충전을 완료하고 다시 일을 시작할 수 있는 상태로 만드는 기능입니다. Auto_charging 기능과 유사하게 자동 충전을 수행하는 기능이지만 차이점은 "idle"상태에서 정해진 시간동안 새로운 job 이 할당되지 않고 timeout 되면 활성화되는 기능입니다. 즉, job 이 완료된 idle 상태에서 일정 시간 뒤 자동으로 자동 충전을 수행하는 기능입니다. 디폴트 설정은 False 입니다.
9	"offline"	Rocon_client 를 정상적으로 종료할 때 정의되는 상태입니다.

4.5.2. status_toggle 상태 정의

- 아래에 정의된 세부 상태로 구성됩니다. 여러 상태 값이 배열 형태로 구성될 수 있습니다.

	Status	Description
1	"error"	Rocon_client 가 job 수행을 정상으로 수행하지 못하거나(로봇 내부의 SW/HW 적인 결함) Concert 와의 동기화에 문제(네트워크 이슈, 설정 오류 등)로 정상적인 수행이 불가능한 상태의 정의입니다. 이 경우 해당 error 의 description 등을 참조하여 사용 관리자의 개입이나 유진로봇의 지원이 필요할 수 있습니다. error 의 세부 정보는 RCM 의 Diagnosis / Error Notice 에서 확인할 수 있습니다.
2	"warning"	로봇 내부에서 비 정상적인 상황에 대한 경고로 표시되는 상태입니다. RCM 의 Diagnosis / Warning Notice 에서 확인할 수 있습니다.
3	"undefined_robot"	로봇 name 설정이 정상적이지 않는 상태입니다.
4	"undefined_fms"	로봇과 연동하는 concert (FMS)의 설정이 정상적이지 않는 상태입니다.
5	"undefined_map"	로봇에서 사용하는 맵의 설정이 정상적이지 않는 상태입니다.
6	"emergency_alarm"	로봇이 정의된 응급 상황에 대한 알람(비상 벨소리)을 감지했을 때의 상태 정의입니다. 이 경우 현재 수행중인 job 이 있다면 자체 중단시키고 지정된 대피 장소로 자동 이동하여 알람 상황이 종료될 때까지 "emergency_alarm"상태로

		머무릅니다. 비상벨이 멈추어 응급 상황이 종료되면 "idle"상태로 자동 전환됩니다. 유럽 규격의 상태정의 입니다.
7	"emergency_button"	로봇에 부착된 emergency 버튼을 누를 때(붉은 색으로 점등)의 상태입니다. 이 상태는 비상 상황에서 로봇을 즉시 멈추고자 할 때 사용됩니다. 현재 수행 중인 job 이 있다면 로봇을 즉시 일시 멈춤 상태로 전환됩니다. "emergency_button"상태를 해지하기 위해서는 로봇에 부착된 emergency 버튼을 다시 눌러 붉은 색으로 점등된 상태를 해지하면 "push_start" 상태로 변경되고 사용자가 "Start"버튼을 누르면 비로서 이전 상태로 자동 복귀합니다. Brake 버튼과 마찬가지로 Concert 를 통한 job 의 정상적인 수행이 되지 않을 때 1 차로 "emergency_button" 상태로 설정되었는지 확인하는 것이 좋습니다. 상태 변화의 예는 다음과 같습니다. "idle"상태에서 "emergency_button" 상태가 되고 다시 emergency button 을 눌러 해지하면 "push_start" 상태가 되며 이때 사용자가 "start"버튼을 누르면 비로소 "idle"상태로 돌아갑니다. "busy"상태에서 "emergency_button" 상태가 되고 다시 emergency button 을 눌러 해지하면 "push_start" 상태가 되며 이때 사용자가 "start"버튼을 누르면 비로소 "busy"상태로 돌아가서 기존 수행중인 job 을 수행하고 끝나면 "idle" 상태로 변환됩니다. 상태의 우선 순위는 "emergency_button"> "brake_released"> "push_start"> "manual_mode"> "emergency_alarm"> "기타 상태" 입니다. 만약 "emergency_button"상태에서 brake 버튼을 off 로 만들면 worker 의 상태는 우선순위가 높은 "emergency_button" 상태가 됩니다. "emergency_button"상태가 해지되면 "brake_released"상태로 자동 전환됩니다.
8	"brake_released"	로봇에 부착된 brake 버튼을 off 로 조작할 때의 상태입니다. off 의 의미는 브레이크가 풀려서 사람이 로봇을 수동으로 밀 수 있는 상태입니다. 이 상태에서는 Concert 에서 지시한 명령을 수행하지 못합니다. 즉, Concert 에서 지시한 명령의 자동 수행은 Brake 가 on 된 상태에서만 가능합니다. 따라서 Concert 를 통한 job 의 정상적인 수행이 되지 않을 때 1 차로 "brake_released"되었는지 확인해야 합니다. 사용자가 Brake 를 off(Released)하고 다시 Brake 를 on 할 때는 "brake_released"되기 직전의 상태로 자동 복귀합니다. 상태 변화의 예는 다음과 같습니다. "idle"상태에서 "brake_released"되고 다시 brake 를 on 하면 "idle"상태로 돌아갑니다. "busy"상태에서 "brake_released"되고 다시 brake 를 on 하면 "busy"상태로 돌아가서 기존 수행중인 job 을 수행하고 끝나면 "idle" 상태로 변환됩니다.
9	"manual_mode"	로봇에 부착된 "manual_auto" 스위치가 "manual"로 된 상태입니다. 이

		상태는 수동 혹은 조이스틱으로 로봇을 조작하기 위한 모드이며, 해당 모드에서는 Concer 에서 지시한 명령을 수행하지 않습니다. 로봇의 manual_auto 스위치는 manual 혹은 autonomous 로 스위치 전환 이후 사용자가 "Start"버튼을 눌러야 비로소 해당 모드로 변경됩니다. 사용자가 manual_auto 스위치를 전환 시 바로 "push_start"상태로 바뀝니다. "Start"버튼을 누르지 않고 여러 번 전환하더라도 "push_start"상태로 유지되고 마지막으로 전환시킨 스위치가 manual 이고 이때 "Start"버튼을 누르면 "manual_mode"상태로 바뀝니다.
10	"robot_locked"	로봇이 init 혹은 idle 상태에서 자체 진단에 의한 독립 행동을 할 때 변경되는 상태입니다. 이 상태일 때 모든 사용자 명령의 수행은 제한되고 자체 독립 행동을 완료하면 이전 상태 혹은 idle 상태로 전이됩니다.
11	"soft_paused"	로봇이 move 상태에서 CMD_PAUSE, CMD_PAUSE_JOB 명령으로 paused 될 때 변경되는 상태입니다. CMD_RESUME, CMD_RESUME_JOB 명령으로 해제해야 상태가 변경됩니다. 또한 Start 버튼으로 Pause/Resume 기능이 적용된 로봇(명세로봇)의 경우 물리버튼에 의해서도 resume 처리됩니다.
12	"traffic_paused"	TBD
13	"button_paused"	로봇이 move 상태에서 로봇에 부착된 "Start"버튼을 누를 때 변경되는 상태입니다. 움직이는 동안에만 적용되며 paused 된 상태에서 다시 "Start"버튼을 누르면 resume 됨과 동시에 busy 상태로 전이됩니다. "Start"버튼을 이용한 pause/resume 기능은 각 로봇의 rocon_client 의 feature enable/disable 로 관리됩니다. (명세로봇은 디폴트로 enabled 됩니다) Start 버튼에 의해 paused 된 상태에서 CMD_RESUME, CMD_RESUME_JOB 에 의해서도 resume 처리됩니다. Soft_paused/button_paused 상태에서 "Start"버튼의 LED 는 주기적으로 점등됩니다.
14	"localizing"	Rocon_client 의 mode 가 "service"혹은 "navigate"이고 초기화 과정인 "init" 상태 다음에 로봇의 위치를 자동으로 탐색하는 상태입니다. 로봇이 change_map, relocation, set_position 등의 위치 탐색을 수행할 때 이 상태로 진입하고 위치 탐색이 완료되면 "idle"상태로 전이됩니다. 또한 "explore"모드에서 다시 "service/navigate"모드로 전환될 때도 "init" > "localizing" > "idle"의 상태 전이가 이루어집니다.
15	"push_start"	"emergency_button" 상태해제 직후 "start"버튼을 눌러야 하는 상태와 "manual_auto"스위치를 전환 후 "start"버튼을 눌러야 하는 상태의 정의입니다. 사용자는 "start"버튼을 눌러야 정상적인 상태 전환이 이루어집니다.

		"push_start" 상태에서 "start"버튼의 LED 가 주기적으로 점등됩니다. 만약 로봇이 move 중에 pused 된 상태에서 emergency_button 을 누르고 다시 emergency 상태를 해제하기 위해 start 버튼을 누르면 paused 된 상태도 자동 resume 처리됩니다.
16	"push_cancel"	로봇에 부착된 cancel 버튼을 눌러야 할 경우 변경되는 상태입니다. "push_cancel" 상태에서 'cancel'버튼의 LED 가 주기적으로 점등됩니다.
17	"escaping"	로봇이 엘리베이터를 이용하고 있는 도중 emergency_alarm 등으로 인해 지정된 탈출 위치로 이동할 때의 상태입니다.
18	"suspend"	RCM 의 설정에서 Suspend 기능이 enabled 될 때 이용 가능한 상태입니다. 현재 진행 중인 Job 을 사용자가 취소하거나 로봇 자체의 실패로 중단될 때 로봇은 'suspend'상태로 됩니다. 이 경우 사용자가 RCM 의 설정 메뉴를 통해 직접 resume 하거나 CMD_RESUME_FOR_SUSPEND API 를 통해서 resume 처리를 할 수 있습니다.
19	"busy_self"	CMD_MOVE, CMD_DOCK 과 같이 Concert 를 통해 작업을 생성하지 않고 로봇에 직접 job 을 생성할 때의 상태입니다. 이 경우 status 는 idle_on_hold 이고 status_toggle 에 busy_self 가 추가되며 status_p 는 busy_self 가 됩니다.
20	"required_charging"	발코니의 dock 명령의 working_percent 값을 지정했을 때, 로봇이 도킹스테이션에 접속한 후 해당 working_percent 값까지 충전하는동안 유지되는 상태입니다.
21	"auto_charging"	Rocon_client 의 설정에서 auto_charging 기능이 enabled 할 때 이용 가능한 상태입니다. Auto_charging 기능은 로봇의 충전 상태를 percentage 기준으로 자동 충전 시작과 충전 완료로 일을 시작할 수 있는 기준을 미리 설정해 두고 해당 조건이 맞으면 자동 충전을 진행하거나 자동 충전을 완료하고 다시 일을 시작할 수 있는 상태로 만드는 기능입니다. 가령 로봇이 busy 상태라도 자동 충전 배터리 조건에 이르면 현재 진행 중인 job 을 강제 abort 시키고 지정된 docking station 으로 자동 복귀하고 지정된 충전 조건까지 "auto_charging"상태를 유지합니다. 디폴트 설정은 False 입니다.

4.5.3. status_p 정의

- Rocon_client는 status 와 status_toggle을 고려하여 우선순위가 가장 높은 하나를 status_p 로 정의 합니다.

- v1.2.0까지 status로 사용한 개념입니다.

4.6. 모드 별 사용 가능한 Message

사용모드의 "공통"은 모든 모드에서 사용할 수 있는 Message 유형입니다. 공통된 요청이거나 각 모드에 맞는 요청 파라미터를 사용하고 그에 대한 결과를 받을 수 있습니다.

지정된 모드 전용의 Message 를 다른 모드에서 사용하면 ack 와 result 는 정상적으로 받을 수 있지만 결과의 세부 값은 실패로 확인됩니다.

현재 Draft 로 정의된 Message 들의 msg_body 내용은 상황에 따라 수정될 수 있습니다. 또한 세부 정의 과정 중이므로 TBD 로 언급된 Message 에 대해서는 이후 업데이트되는 문서에서 정리될 예정입니다.

명령	사용 모드	msg_body 필드: name	간단 설명
로그인	공통	login	Rocon Client Service 서버에 Websocket 접속 성공 직후 login 을 해야 합니다. default timeout 시간내에 login 을 하지 않을 경우 자동 disconnect 됩니다. 정상적인 로그인 이후에 모든 command 요청 및 feedback 수신이 가능합니다.
로그아웃	공통	logout	Rocon Client Service 서버에서 로그아웃 합니다. 정상적인 로그아웃 되면 서버에서 자동 disconnect 합니다.
모드변경	공통	change_mode	navigate 혹은 explore 모드로 변경을 할 때 사용합니다.
시스템 재시작	공통	reboot	시스템 SW 를 재 시작하고자 할 때 사용합니다.
상태요청	공통	get_status	현재 모드의 다양한 상태 정보를 요청합니다.
설정요청	공통	get_configs	현재 모드에서 세부 설정 정보 값을 요청합니다.
설정변경	공통	set_configs	현재 모드에서 세부 설정을 변경할 때 사용합니다.
피드백설정	공통	set_feedback	현재 모드에서의 자동으로 피드백 받을 상태 정보를 설정할 때 사용합니다.
리소스 정보 요청	service	get_resources	FMUI 에 등록된 각종 리소스 정보를 요청할 때 사용합니다. 리소스 정보의 세부 type 으로 로봇이 이동 가능한 위치들인 "waypoints", 로봇이 충전할 수 있는 충전소 정보인 "charging-stations" 등이 있습니다.
Job 생성	service	compose_job	Concert 에 Job 생성을 요청할 때 사용합니다.
Job 취소	service	cancel_job	Concert 에 현재 실행 중인 Job 실행을 중단할 때 사용합니다.
Job 일시정지	service	pause_job	Job 의 세부 명령 중 로봇의 수행 명령을 일시 중지할 때 사용합니다.

Job 재개	service	resume_job	Job 의 세부 명령 중 로봇의 수행 명령이 일시 정지 상태라면 이를 해지하고 재개할 때 사용합니다.
지도변경	service, navigate	change_map	로봇의 현재 위치하고 있는 맵을 새로 지정된 맵으로 변경할 때 사용합니다.
위치보정	service, navigate	relocation	로봇의 지도상의 위치를 보정하기 위해 사용합니다.
장애물 정보 제거	service, navigate	clear_obstacle	현재 사용중인 지도 상에 센서로부터 취득된 장애물 정보를 소거합니다
이동	navigate	move	로봇의 현재 위치에서 지정된 위치로 이동 명령할 때 사용합니다.
멈춤	navigate	stop	로봇이 수행하고 있는 현재 작업을 중단할 때 사용합니다.
일시정지	navigate	pause	현재 로봇이 수행하고 있는 작업을 일시 정지할 때 사용합니다.
재개	navigate	resume	현재 로봇의 수행하고 있는 작업이 일시 정지 상태라면 이를 해지하고 재개할 때 사용합니다.
충전	navigate	dock	지정된 충전소(dock station)로 도킹 명령할 때 사용합니다. (Service API v1.2.3 부터 지원하지 않습니다)
충전	navigate	charge	지정된 충전소(dock station)로 도킹 명령할 때 사용합니다. (Service API v1.2.3 신규 추가)
주차	navigate	park	지정된 marker, DIM, Robostop(측면주차)에 정밀 주차할 때 사용합니다. (Service API v1.2.3 신규 추가)
주차해제	navigate	unpark	Marker, Charging station, DIM, Robostop 으로부터 파킹을 해제할 때 사용합니다. (Service API v1.2.3 신규 추가)
	explore		

<표 4-5 모드별 사용 가능한 Message>

5. Message 유형별 세부 설명(Custom Client to Rocon Client Service)

5.

5.1. 로그인: login(Command)

- 간략히 CMD_LOGIN으로 칭합니다.
- Rocon Client Service Server에 접속한 직후 default timeout(1분)내에 로그인 해야 합니다. 로그인 되지 않으면 서버는 자동 disconnect 처리합니다.
- 정상적인 로그인 이후 모든 명령과 피드백 정보를 수신할 수 있습니다.
- 정상적인 로그인 시 결과에 기본 정보들이 포함되며, 이후 주기적인 feedback 정보를 받게 됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"login"	String
params		Object
login_id	로그인을 위한 아이디입니다. FMS 등록시 사용하는 worker의 name을 사용합니다.	String
login_pwd	로그인을 위한 패스워드입니다. (현재 버전에서 지원되지 않습니다.)	String

<표 5-1 login Message의 msg_body 구성>

login Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "login",
    "params": {
      "login_id": "gocart_blcho",
      "login_pwd": "xxx"
    }
  },
  "msg_uuid": "login_1",
  "msg_ver": "1.0"
}
```

login 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "login",
    "result": "success",
    "error_info": null,
    "data_contents_type": "login_info",
    "data_contents": {
      "worker_info": {
        "type": "robot",
        "name": "real_blcho",
        "uuid": "real_blcho",
        "task_name": "CSSD Task Test",
        "task_description": "CSSD Task Test",
        "status": "idle",
        "status_p": "idle",
        "status_toggle": [],
        "robot_width": 0.51,
        "robot_length": 0.73,
        "map": {
          "id": "5f4f2de9429b4a0012d40c54",
          "name": "3F"
        }
      },
      "pose": {
        "theta": 1.6015,
        "x": 4.9737,
        "y": 0.3961
      },
      "velo": {
        "velo_dx": 0.0,
        "velo_dy": 0.0,
        "velo_dz": 0.0
      },
      "battery": {
        "battery_level": 30.0,
        "now_charging": false,
        "charge_source": "docker"
      },
      "brake_released": false,
      "paused": false,
    }
  }
}
```

```

    "emergency_alarm": false,
    "emergency_button_pressed": false,
    "manual_mode": false,
    "in_charging": false,
    "robot_width": 0.51,
    "robot_length": 0.73,
    "robot_state": "ready",
    "robot_navi_mode": "navigate",
    "virtual_worker": true,
    "path_plan": null,
    "odometry": null
  }
},
"msg_uuid": "login_1",
"msg_ver": "1.0"
}

```

5.2. 로그아웃: logout(Command)

- 간략히 CMD_LOGOUT으로 칭합니다.
- Rocon Client Service Server에서 로그 아웃할 때 사용합니다.
- 정상적인 로그아웃 이후 서버는 자동 disconnect 처리합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"logout"	String

<표 5-2 login Message 의 msg_body 구성>

logout Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "logout"
  },
  "msg_uuid": "logout_1",
  "msg_ver": "1.0"
}

```

logout 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "logout",
    "result": "success",
    "error_info": null,
    "data_contents_type": null,
    "data_contents": null
  },
  "msg_uuid": "logout_1",
  "msg_ver": "1.0"
}
```

5.3. 모드변경: change_mode(Command)

- 간략히 CMD_CHANGE_MODE로 표시합니다.
- "service", "explore" 2가지 모드 중 한가지를 선택하는 Message입니다.
- 모드 변경시 이전 모드에서의 작업은 모두 자동 강제 종료 후 진행됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"change_mode"	String
params		Object
mode	mode 는 rocon_client 가 부팅 시 'boot' 로 설정되며 자동으로 "service"로 변경됩니다. 단 최초에 맵이 없는 상태라면 자동으로 "explore" 모드로 변경됩니다. 이후 "service", "explore" 중 하나를 필요에 의해 전환 선택 가능합니다. * 모드 설정이 완료된 result 값이 정상 적일 때 해당 모드의 운영이 가능합니다. * rocon_client 기동 될 때 생성된 맵이 존재하지 않는 경우 자동으로 explore 모드로 전환 시작됩니다. * rocon_client 기동 될 때 생성된 맵이 존재한다면 자동으로 service 모드로 시작합니다. * service 모드에서 explore 모드로 전환에 성공하면 사용자는 바로 맵을 그릴 수 있는 상태입니다. 이때 joystick 을 이용하여 로봇을 움직이면 맵 생성이 진행됩니다. 맵의 업데이트 과정 정보는 feedback event 에 맵 생성 데이터 정보를 참고하세요.	String

<표 5-3 change_mode Message 의 msg_body 구성>

change_mode Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "change_mode",
    "params": {
      "mode": "service"
    }
  },
  "msg_uuid": "change_mode_1",
  "msg_ver": "1.0"
}
```

change_mode 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "change_mode",
    "result": "success",
    "data_contents": {
      "current_mode": "service"
    }
  },
  "msg_uuid": "change_mode_1",
  "msg_ver": "1.0"
}
```

5.4. 피드백설정: set_feedback(Command)

- 간략히 CMD_SET_FEEDBACK으로 칭합니다.
- Feedback Message는 사용자 클라이언트에서 모드 설정을 완료한 이후 주기적으로 서버에서 클라이언트로 전달됩니다.
- 피드백설정은 이 피드백 타임 주기를 조정할 때 이용하는 명령입니다.

msg_body 필드명	설명 및 설정 값	자료형
name	"set_feedback"	String
params		Object
interval_sec	피드백 Message 수신 간격(초)를 설정합니다. (디폴트 0.3 초)	Float

<표 5-4 set_feedback Message 의 msg_body 구성>

set_feedback Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "set_feedback",
    "params": {
      "interval_sec": 2
    }
  },
  "msg_uuid": "set_feedback_1",
}
```

```
{
  "msg_ver": "1.0"
}
```

set_feedback 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "set_feedback",
    "result": "success"
  },
  "msg_uuid": "set_feedback_1",
  "msg_ver": "1.0"
}
```

5.5. 리소스정보 요청: get_resources(Command)

- 간략히 CMD_GET_RESOURCES로 표시합니다.
- FMUI에서 맵에서 설정한 리소스 정보들을 요청할 때 사용하는 명령입니다.
- 대표적인 리소스 정보는 다음과 같습니다.

Resource type	설명
"waypoints"	FMUI 상에 지정된 로봇이 이동 가능한 지점입니다.
"charging-stations"	FMUI 상에 지정된 로봇이 충전가능한 충전소입니다. (Service API v1.2.3 부터 지원하지 않습니다)
"markers"	FMUI 상에 지정된 로봇이 충전가능한 충전소 혹은 마커입니다.
"DIM"	FMUI 상에 지정된 리소스 DIM 입니다.
"Robostop"	FMUI 상에 지정된 리소스 Robostop 입니다. (Side parking)
"maps"	FMUI 상에 등록된 Map 정보입니다.
"zones"	FMUI 상에 등록된 Map 상의 특정 영역 정보입니다. zones 는 세부적으로 "prohibit"(금지구역), "escape"(탈출지역) 등의 세부 타입이 존재합니다.
"autodoors"	FMUI 상에 등록된 Map 상의 자동문 영역 정보입니다.
"teleporter-gates"	FMUI 상에 등록된 Map 상의 엘리베이터 영역 정보입니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_resources"	String
params		Object
type	원하는 리소스 타입 명칭을 사용합니다.	String
map_id	원하는 리소스가 존재하는 map 을 지정합니다.	String

	map_id 값이 null 이거나 필드가 없으면 전체 리소스를 반환합니다.	
--	---	--

<표 5-5 get_resources Message 의 msg_body 구성>

get_resources Message 예시(충전소 정보 요청)

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_resources",
    "params": {
      "type": "charging-stations"
    }
  },
  "msg_uuid": "get_resources_1",
  "msg_ver": "1.0"
}
```

get_resources 결과 Message 예시(충전소 정보 결과)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_resources",
    "result": "success",
    "data_contents_type": "charging-stations",
    "data_contents":
      [# 해당 리소스 정보를 배열 구조로 나열합니다.
      {
        "type": "Gocart_120",
        "description": null,
        "resource_active": true,
        "resource_type": "ChargingStation",
        "map": "5e1816271176ba0012f76b76",
        "pose": {
          "x": -2.640625,
          "y": -1.78125,
          "theta": 6.2397344117880555
        },
        "marker_value": 10,
        "marker_type": 'gocart_ar",
        "name": "1st Cafe station",
        "created_at": '2020-09-10T08:09:32.605Z",
      }
    ]
  }
}
```

```

        "updated_at": "2020-09-10T08:09:32.605Z",
        "id": "5f55d8e3b352870019eda55f"
    },

    {
        "type": "Gocart_120",
        "description": null,
        "resource_active": true,
        "resource_type": "ChargingStation",
        "map": "5e26a1331176ba0012f76b94",
        "pose": {
            "x": -34.5808755884387,
            "y": -22.973960876464844,
            "theta": 1.5273454314033659
        },
        "marker_value": 14,
        "marker_type": "gocart_ar",
        "name": "test_station",
        "created_at": "2020-09-10T08:09:32.605Z",
        "updated_at": "2020-09-10T08:09:32.605Z",
        "id": "5f597e4d419a4400124e5c71"
    }
]
},
"msg_uuid": "get_resources_1",
"msg_ver": "1.0"
}

```

get_resources Message 예시(maps 정보 요청)

```

{
    "msg_type": "command",
    "msg_body": {
        "name": "get_resources",
        "params": {
            "type": "maps"
        }
    }
},
"msg_uuid": "get_resources_1",
"msg_ver": "1.0"
}

```

get_resources 결과 메시지 예시(maps 정보 결과)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_resources",
    "result": "success",
    "error_info": null,
    "data_contents_type": "maps",
    "data_contents": [
      {
        "original": {
          "gridmap": "1599024567447/origin/grid.png",
          "nvmap": "1599024567447/origin/visual_map.nvmap",
          "metadata": "1599024567447/origin/grid_cfg.grid"
        },
        "files": {
          "gridmap": "1599024567447/gridmap1599024567792_flipped.png",
          "nvmap": "1599024567447/origin/visual_map.nvmap",
          "metadata": "1599024567447/origin/grid_cfg.grid",
          "monochrome": "1599024567447/monochrome1599024567819_flipped.bmp"
        },
        "metadata": {
          "gridmap": {
            "origin": {
              "x": 694,
              "y": 1155
            },
            "width": 1299,
            "height": 2086,
            "scale_m2px": 32
          },
          "monochrome": {
            "width": 623,
            "height": 1000
          }
        },
        "description": "map generated by mapUploader at 2020-09-02T05:29:27.447Z",
        "_id": "5f4f2db7429b4a0012d40c52",
        "name": "1F",

```

```

    "map_package": "1599024567447",
    "created_at": "2020-09-02T05:29:27.812Z",
    "updated_at": "2020-09-14T06:50:59.021Z",
    "site": "5f4f3f57429b4a0012d40c58",
    "id": "5f4f2db7429b4a0012d40c52"
  },
  {
    "original": {
      "gridmap": "1599024617031/origin/grid.png",
      "nvmap": "1599024617031/origin/visual_map.nvmap",
      "metadata": "1599024617031/origin/grid_cfg.grid"
    },
    "files": {
      "gridmap": "1599024617031/gridmap1599024617564_flipped.png",
      "nvmap": "1599024617031/origin/visual_map.nvmap",
      "metadata": "1599024617031/origin/grid_cfg.grid",
      "monochrome": "1599024617031/monochrome1599024617585_flipped.bmp"
    },
    "metadata": {
      "gridmap": {
        "origin": {
          "x": 2540,
          "y": 545
        },
        "width": 2926,
        "height": 1496,
        "scale_m2px": 32
      },
      "monochrome": {
        "width": 1000,
        "height": 511
      }
    },
    "description": "map generated by mapUploader at 2020-09-02T05:30:17.031Z",
    "_id": "5f4f2de9429b4a0012d40c54",
    "name": "3F",
    "map_package": "1599024617031",
    "created_at": "2020-09-02T05:30:17.579Z",
    "updated_at": "2020-10-06T03:46:02.211Z",
    "site": "5f4f3f57429b4a0012d40c58",

```

```

      "id": "5f4f2de9429b4a0012d40c54"
    },
    {
      "original": {
        "gridmap": "1599024624765/origin/grid.png",
        "nvmap": "1599024624765/origin/visual_map.nvmap",
        "metadata": "1599024624765/origin/grid_cfg.grid"
      },
      "files": {
        "gridmap": "1599024624765/gridmap1599024625088_flipped.png",
        "nvmap": "1599024624765/origin/visual_map.nvmap",
        "metadata": "1599024624765/origin/grid_cfg.grid",
        "monochrome": "1599024624765/monochrome1599024625113_flipped.bmp"
      },
      "metadata": {
        "gridmap": {
          "origin": {
            "x": 673,
            "y": 469
          },
          "width": 1931,
          "height": 1379,
          "scale_m2px": 32
        },
        "monochrome": {
          "width": 1000,
          "height": 714
        }
      },
      "description": "map generated by mapUploader at 2020-09-02T05:30:24.765Z",
      "_id": "5f4f2df1429b4a0012d40c55",
      "name": "4F",
      "map_package": "1599024624765",
      "created_at": "2020-09-02T05:30:25.106Z",
      "updated_at": "2020-09-14T06:50:51.380Z",
      "site": "5f4f3f57429b4a0012d40c58",
      "id": "5f4f2df1429b4a0012d40c55"
    }
  ]
},

```



```
"msg_uuid": "result",  
"msg_ver": "1.0"  
}
```

get_resources Message 예시(waypoints 정보 요청)

```
{  
  "msg_type": "command",  
  "msg_body": {  
    "name": "get_resources",  
    "params": {  
      "type": "waypoints"  
    }  
  },  
  "msg_uuid": "get_resources_1",  
  "msg_ver": "1.0"  
}
```

get_resources 결과 메시지 예시(waypoints 정보 결과)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_resources",
    "result": "success",
    "error_info": null,
    "data_contents_type": "waypoints",
    "data_contents": [
      {
        "description": "",
        "resource_active": true,
        "resource_type": "Location",
        "_id": "5f4f4dca228e35001204bf6e",
        "pose": {
          "x": -18.346573175413404,
          "y": -1.1669714277001972,
          "theta": 4.680142097949435
        },
        "name": "3F_Infodesk",
        "map": "5f4f2de9429b4a0012d40c54",
        "type": "waypoint",
        "created_at": "2020-09-02T07:46:18.204Z",
        "updated_at": "2020-09-16T06:44:34.805Z",
        "id": "5f4f4dca228e35001204bf6e"
      },
      {
        "description": "",
        "resource_active": true,
        "resource_type": "Location",
        "_id": "5f4f4dda228e35001204bf6f",
        "pose": {
          "x": -20.781349481565748,
          "y": -11.943216544887697,
          "theta": -0.09011597479384803
        },
        "name": "3F_ANS",
        "map": "5f4f2de9429b4a0012d40c54",
        "type": "waypoint",
        "created_at": "2020-09-02T07:46:34.933Z",
```

```

        "updated_at": "2020-09-23T01:23:10.637Z",
        "id": "5f4f4dda228e35001204bf6f"
    },
    {
        "description": "",
        "resource_active": true,
        "resource_type": "Location",
        "_id": "5f4f4df7228e35001204bf70",
        "pose": {
            "x": -8.995472253782538,
            "y": -12.45594809701172,
            "theta": 3.141592653589793
        },
        "name": "3F_Coretech",
        "map": "5f4f2de9429b4a0012d40c54",
        "type": "waypoint",
        "created_at": "2020-09-02T07:47:03.116Z",
        "updated_at": "2020-09-23T01:22:35.653Z",
        "id": "5f4f4df7228e35001204bf70"
    }
]
},
"msg_uuid": "result",
"msg_ver": "1.0"
}

```

get_resources Message 예시(zones 정보 요청)

```

{
    "msg_type": "command",
    "msg_body": {
        "name": "get_resources",
        "params": {
            "type": "zones",
            "map_id": "5f4f2de9429b4a0012d40c54"
        }
    },
    "msg_uuid": "current_map_zones_1",
    "msg_ver": "1.0"
}

```

get_resources 결과 Message 예시(zones 정보 요청)

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "get_resources",
    "result": "success",
    "data_contents_type": "zones",
    "data_contents": [
      {
        "param_sets": [],
        "description": null,
        "resource_active": true,
        "resource_type": "Zone",
        "_id": "5f4f4e30228e35001204bf71",
        "name": "Conference_Window",
        "type": "prohibit",
        "areas": [
          {
            "x": 0.1774203,
            "y": -7.19846985,
            "theta": -0.021304309321590553,
            "width": 4.058125,
            "height": 12.9066435
          }
        ],
        "map": "5f4f2de9429b4a0012d40c54",
        "worker_params": {
          "designated": {
            "pose": []
          }
        },
        "created_at": "2020-09-02T07:48:00.219Z",
        "updated_at": "2020-09-02T07:48:00.219Z",
        "id": "5f4f4e30228e35001204bf71"
      },
      {
        "param_sets": [],
        "description": null,
        "resource_active": true,

```

```

    "resource_type": "Zone",
    "_id": "5f59cd72d25ef600139c7ebe",
    "name": "zone-4pamq",
    "type": "waiting",
    "areas": [
      {
        "x": -8.81997553,
        "y": -1.22971985,
        "theta": 0,
        "width": 3.25,
        "height": 3.95
      }
    ],
    "map": "5f4f2de9429b4a0012d40c54",
    "worker_params": {
      "designated": {
        "pose": []
      }
    },
    "created_at": "2020-09-10T06:53:38.781Z",
    "updated_at": "2020-09-10T06:53:38.781Z",
    "id": "5f59cd72d25ef600139c7ebe"
  },
  {
    "param_sets": [],
    "description": "3F_lidar_test_zone",
    "resource_active": true,
    "resource_type": "Zone",
    "_id": "5f695cb01021920034fa6316",
    "name": "Lidar_Test_Area",
    "type": "prohibit",
    "areas": [
      {
        "x": -3.246387656675032,
        "y": -12.660242122354028,
        "theta": 6.256753147639789,
        "width": 3.25,
        "height": 3.0066387571134565
      }
    ]
  },

```

```

    "map": "5f4f2de9429b4a0012d40c54",
    "worker_params": {
      "designated": {
        "pose": []
      }
    },
    "created_at": "2020-09-22T02:08:48.471Z",
    "updated_at": "2020-09-22T02:09:20.715Z",
    "id": "5f695cb01021920034fa6316"
  },
  {
    "param_sets": [],
    "description": null,
    "resource_active": true,
    "resource_type": "Zone",
    "_id": "5fb1fa8bfadbe30040f96727",
    "name": "escape_zone",
    "type": "escape",
    "areas": [
      {
        "x": -2.82248098,
        "y": -3.22399795,
        "theta": -0.02440417,
        "width": 1.65,
        "height": 2.5
      }
    ],
    "map": "5f4f2de9429b4a0012d40c54",
    "worker_params": {
      "designated": {
        "pose": [
          {
            "x": -2.2879707301107715,
            "y": 0.4126800534338031,
            "theta": 4.69
          }
        ]
      }
    },
    "created_at": "2020-11-16T04:05:31.089Z",

```

```

        "updated_at": "2020-11-16T04:05:31.089Z",
        "id": "5fb1fa8bfadbe30040f96727"
    }
]
},
"msg_uuid": "current_map_zones_1",
"msg_ver": "1.0"
}

```

get_resources Message 예시(autodoors 정보 요청)

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "get_resources",
    "params": {
      "type": "autodoors",
      "map_id": "5f4f2de9429b4a0012d40c54"
    }
  },
  "msg_uuid": "current_map_autodoors",
  "msg_ver": "1.0"
}

```

get_resources 결과 Message 예시(autodoors 정보 요청)

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_resources",
    "result": "success",
    "data_contents_type": "autodoors",
    "data_contents": [
      {
        "resource_waitings": [
          "5f684f161021920034fa6301",
          "5f684f161021920034fa6302"
        ],
        "description": "3F_Autodoor",
        "resource_active": true,
        "resource_type": "Autodoor",
        "_id": "5f684f161021920034fa6303",
        "name": "3F_Autodoor",
        "areas": [
          {
            "x": -29.78796373,
            "y": 0.66476793,
            "theta": 6.23859605,
            "width": 1.9,
            "height": 3.85149742
          }
        ],
        "map": "5f4f2de9429b4a0012d40c54",
        "aligns": [
          {
            "x": -30.88206683505114,
            "y": 0.7073750215419565,
            "theta": 6.238596051924514
          },
          {
            "x": -28.68087766579825,
            "y": 0.6223226350622042,
            "theta": 3.09
          }
        ]
      }
    ]
  }
}
```



```

        "properties": {
            "connection": {
                "ip": "192.168.10.212",
                "port": 6722,
                "group_id": 1
            },
            "vendor": "sr201"
        },
        "created_at": "2020-09-21T06:58:30.284Z",
        "updated_at": "2020-10-21T05:57:19.446Z",
        "id": "5f684f161021920034fa6303"
    }
]
},
"msg_uuid": "current_map_autodoors",
"msg_ver": "1.0"
}

```

get_resources Message 예시(teleporter-gates 정보 요청)

```

{
    "msg_type": "command",
    "msg_body": {
        "name": "get_resources",
        "params": {
            "type": "teleporter-gates",
            "map_id": "5f4f2de9429b4a0012d40c54"
        }
    },
    "msg_uuid": "current_map_teleporter_gates",
    "msg_ver": "1.0"
}

```

get_resources 결과 Message 예시(teleporter-gates 정보 요청)

```

{
    "msg_type": "result",
    "msg_body": {
        "name": "get_resources",
        "result": "success",
        "data_contents_type": "teleporter-gates",
        "data_contents": [

```

```

{
  "aligns": {
    "entries": [
      {
        "x": -33.61041793823242,
        "y": -3.53125,
        "theta": 4.666229871030253
      }
    ],
    "exits": [
      {
        "x": -33.610406494140626,
        "y": -3.634167938232423,
        "theta": 1.526229871030253
      }
    ]
  },
  "networks": [],
  "resource_waitings": [
    "5f4f4da2228e35001204bf6b",
    "5f681ff4d25ef600139c7ed7"
  ],
  "description": null,
  "resource_active": true,
  "resource_type": "TeleporterGate",
  "_id": "5f4f3679429b4a0012d40c56",
  "name": "3F_elv",
  "map": "5f4f2de9429b4a0012d40c54",
  "area": {
    "x": -33.65029765,
    "y": -4.62755909,
    "theta": -1.60377013,
    "width": 1.95627525,
    "height": 1.9015187
  },
  "properties": {
    "floor_id": "4",
    "floor_name": "3F",
    "doorNumbers": [
      0
    ]
  }
}

```

```

        ]
      },
      "parameters": {
        "marker_id": 0,
        "marker_pose": {
          "x": 0,
          "y": 0,
          "theta": 0
        }
      },
      "created_at": "2020-09-02T06:06:49.766Z",
      "updated_at": "2020-09-23T01:16:38.421Z",
      "teleporter": "5f5042b2228e35001204bfaa",
      "id": "5f4f3679429b4a0012d40c56"
    }
  ]
},
"msg_uuid": "current_map_teleporter_gates",
"msg_ver": "1.0"
}

```

5.6. Task 정의 요청: get_task_definition(Command)

- 간략히 CMD_GET_TASK_DEFINITION으로 표시합니다.
- 로봇이 Concert를 통해 수행할 수 있는 작업을 생성하기 위해 정의된 스키마입니다. 각 명령은 actions 배열에 정의되어 있습니다.
- CMD_COMPOSE_JOB으로 다중 수행 명령을 만들때 이 스키마를 참조하여 구성합니다.

Get_task_definition Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "get_task_definition"
  },
  "msg_uuid": "get_task_definition _1",
  "msg_ver": "1.0"
}

```

Get_task_definition 결과

>> RCM 의 Service API Test 에서 직접 확인하세요.

5.7. Job생성: compose_job(Command)

- 간략히 CMD_COMPOSE_JOB으로 표시합니다.
- CMD_CREATE_JOB은 더 이상 사용되지 않으므로 대신 이 명령을 사용해 주세요.
- Concert를 통해 로봇이 수행하는 새로운 Job을 만들도록 요청할 때 사용하는 명령입니다.
- Job 만들기 요청할 때 세부 Instruction(단위 명령) 들의 조합으로 구성할 수 있습니다.
- Rocon Service는 Concert에 Job생성 요청 결과를 확인하고 사용자에게 결과 메시지를 전송합니다. 이후 로봇은 Job을 받아서 수행하게 됩니다. 수행중인 Job의 진행 사항은 사용자가 수신하는 피드백 메시지의 status, job_id, 로봇의 이동 위치 정보 등을 바탕으로 수행 중임을 알 수 있습니다.
- RCM의 Service API Test에서 제공되는 요청 메시지 내용은 샘플이므로 정상적인 실행을 위해서는 각 명령의 func_name, args의 내용을 CMD_GET_TASK_DEFINITION의 형식과 도메인 범위내의 리소스를 정확히 지정해야 정상 실행됩니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"compose_job"	String
params			Object
	insts	원하는 원하는 단위명령을 배열 구조로 지정합니다. FMS 의 WS-Open API 의 reqComposeJob 의 params 의 insts 와 동일한 구조로 insts 필드를 완성하세요. CMD_GET_TASK_DEFINITION 으로 각 명령의 스키마를 확인 할 수 있습니다.	Array

<표 5-6 comose_job Message 의 msg_body 구성>

compose_job Message 예시

```
{
  "msg_type": "command",
```

```

"msg_body": {
  "name": "compose_job"
  "params": {
    "insts": [# 배열 구조로 만들어 나열합니다.
      {
        "func_name": "move",  # 첫번째 move 명령
        "args": {
          "destination": "6864fb691c7020996ab6b7c5",
          "finishing_timeout": -1
        }
      },
      {
        "func_name": "move",  # 두번째 move 명령
        "args": {
          "destination": "6864fb771c7020996ab6b842",
          "finishing_timeout": -1
        }
      },
      {
        "func_name": "dock",   # 세번째 dock 명령
        "args": {
          "resource": "6864fbbe1c7020996ab6ba58",
          "charge_switch": false,
          "method": "marker",
          "working_percent": -1,
          "do_move": true,
          "finishing_timeout": -1
        }
      }
    ]
  }
},
"msg_uuid": "compose_job_1",
"msg_ver": "1.0"
}

```

compose_job 결과 Message 예시

```

{
  "msg_type": "result",

```

```

"msg_body": {
  "name": "compose_job",
  "result": "success",
  "data_contents": {
    "job_id": "5f2a45a70deb9800196c9003"  #생성된 job 의 id 를 반환합니다.
  }
},
"msg_uuid": "compose_job_1",
"msg_ver": "1.0"
}

```

5.8. Job취소: cancel_job(Command)

- 간략히 CMD_CANCEL_JOB으로 표시합니다.
- "compose_job"으로 생성 실행한 job을 취소할 때 사용하는 명령입니다.
- "compose_job" 명령으로 받은 job_id를 지정해야 하며, 현재 해당 job_id로 수행 중이 아니면 result Message는 failure를 리턴 받습니다.

msg_body 필드명	설명 및 설정 값	자료형
name	"cancel_job"	String
params		Object
job_id	compose_job 생성시 받은 job_id 를 지정합니다.	String

<표 5-7 cancel_job Message 의 msg_body 구성>

cancel_job Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "cancel_job",
    "params": {
      "job_id": "5f2a45a70deb9800196c9003"
    }
  },
  "msg_uuid": "cancel_job_1",
  "msg_ver": "1.0"
}

```

cancel_job 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "cancel_job",
    "result": "success",
    "data_contents": {
      "job_id": "5f2a45a70deb9800196c9003"
    }
  },
  "msg_uuid": "cancel_job_1",
  "msg_ver": "1.0"
}
```

5.9. Job일시정지: pause_job(Command)

- 간략히 CMD_PAUSE_JOB으로 표시합니다.
- "compose_job"으로 생성 실행한 job 수행 중 로봇을 일시 정지할 때 사용하는 명령입니다.
- "compose_job" 명령으로 받은 job_id를 지정해야 하며, 현재 해당 job_id로 수행 중이 아니면 result Message는 failure를 리턴 받습니다.
- Job 중에서 현재 로봇에 지정된 단위 명령에서의 로봇 기준의 일시 정지입니다. 가령 move 단위 명령 수행 중에 pause_job을 받으면 로봇의 수행이 일시 정지되므로 전체 job의 진행 또한 일시 정지됩니다.

msg_body 필드명	설명 및 설정 값	자료형
name	"pause_job"	String
params		Object
job_id	compose_job 생성시 받은 job_id 를 지정합니다.	String

<표 5-8 pause_job Message 의 msg_body 구성>

pause_job Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "pause_job",
    "params": {
      "job_id": "5f2a45a70deb9800196c9003"
    }
  },
}
```

```
{
  "msg_uuid": "pause_job_1",
  "msg_ver": "1.0"
}
```

pause_job 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "pause_job",
    "result": "success",
    "data_contents": {
      "job_id": "5f2a45a70deb9800196c9003"
    }
  },
  "msg_uuid": "pause_job_1",
  "msg_ver": "1.0"
}
```

5.10. Job재개: resume_job(Command)

- 간략히 CMD_RESUME_JOB으로 표시합니다.
- "pause_job"으로 일시 정지된 job 수행을 재개할 때 사용하는 명령입니다.
- "compose_job" 명령으로 받은 job_id를 지정해야 하며, 현재 해당 job_id로 수행 중이 아니면 result Message는 failure를 리턴 받습니다.

msg_body 필드명	설명 및 설정 값	자료형
name	"resume_job"	String
params		Object
job_id	compose_job 생성시 받은 job_id 를 지정합니다.	String

<표 5-9 resume_job Message 의 msg_body 구성>

resume_job Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "resume_job",
    "params": {
      "job_id": "5f2a45a70deb9800196c9003"
    }
  }
}
```



```
}  
,  
"msg_uuid": "resume_job_1",  
"msg_ver": "1.0"  
}
```

resume_job 결과 Message 예시

```
{  
  "msg_type": "result",  
  "msg_body": {  
    "name": "resume_job",  
    "result": "success",  
    "data_contents": {  
      "job_id": "5f2a45a70deb9800196c9003"  
    }  
  },  
  "msg_uuid": "resume_job_1",  
  "msg_ver": "1.0"  
}
```

5.11. 프리 셋 조회: get_presets(Command)

- 간략히 CMD_GET_PRESETS로 표시합니다.
- Balcony에서 로봇의 실행을 위해 다수의 action 조합으로 만든 Job을 프리 셋으로 저장할 수 있습니다. 이 프리 셋을 Service API를 통해 조회할 때 사용하는 명령입니다.
- 조회된 프리 셋의 리스트를 반환합니다. (배열 구조)
- 조회된 프리 셋 리스트내의 "title", "id"의 값이 프리 셋을 선택하는 주요 키입니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_presets"	String

<표 5-10 get_presets Message 의 msg_body 구성>

get_presets Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_presets"
  },
  "msg_uuid": "get_presets",
  "msg_ver": "1.0"
}
```

get_presets 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_presets",
    "result": "success",
    "data_contents": [
      {
        "description": "sfsfds",
        "_id": "603ddaceab7a65002a32afe7",
        "title": "New Preset: 2021-03-02 3:27:15 pm",
        "owner": "602e133ecd70460035b8b168",
        "options": {
          "instructions": [
            {
              "id": "999c30aeb4ff6209bc3c1f1b36e05a88",
              "action": {
```

```
"args": [  
  {  
    "key": "destination",  
    "type": "string",  
    "value": "5f4f4dca228e35001204bf6e",  
    "value_alias": {  
      "alias": "3F_Infodesk",  
      "value": "5f4f4dca228e35001204bf6e"  
    },  
    "options": {  
      "user_input": false  
    },  
    "isParamArgs": false  
  },  
  {  
    "key": "goal_finishing",  
    "type": "boolean",  
    "value": true,  
    "value_alias": {  
      "alias": "True",  
      "value": true  
    },  
    "options": {  
      "user_input": false  
    },  
    "isParamArgs": false  
  },  
  {  
    "key": "finishing_timeout",  
    "type": "number",  
    "value": 30,  
    "value_alias": {  
      "alias": "30",  
      "value": 30  
    },  
    "options": {  
      "max": 300,  
      "min": 0,  
      "user_input": true  
    },  
  },  
]
```

```

        "isParamArgs": false
      },
    ],
    "func_name": "move"
  },
  "next": null
}
],
"task": "60337a0ede1d14e2a9ca02f8",
"rules": [
  {
    "rule": "compelWorker",
    "args": {
      "workerId": "602e3bbbdd876a002aba8ddb"
    }
  }
],
"events": []
},
"shared": [],
"created_at": "2021-03-02T06:27:26.401Z",
"updated_at": "2021-03-02T06:27:35.732Z",
"id": "603ddaceab7a65002a32afe7"
},
{
  "description": "blcho_preset_1",
  "_id": "603f2ddcff14a3002a541035",
  "title": "blcho_preset_1",
  "owner": "602e133ecd70460035b8b168",
  "options": {
    "instructions": [
      {
        "id": "f111915a7f5c5ca7d44a72e7f5b9a96f",
        "action": {
          "args": [
            {
              "key": "destination",
              "type": "string",
              "value": "5f4f4eb3228e35001204bf74",
              "value_alias": {

```

```

        "value": "5f4f4eb3228e35001204bf74",
        "alias": "3F_302"
    },
    "options": {
        "user_input": false
    },
    "isParamArgs": false
},
{
    "key": "goal_finishing",
    "type": "boolean",
    "value": true,
    "value_alias": {
        "alias": "True",
        "value": true
    },
    "options": {
        "user_input": false
    },
    "isParamArgs": false
},
{
    "key": "finishing_timeout",
    "type": "number",
    "value": 30,
    "value_alias": {
        "alias": "30",
        "value": 30
    },
    "options": {
        "max": 300,
        "min": 0,
        "user_input": true
    },
    "isParamArgs": false
}
],
"func_name": "move"
},
"next": "00467ee724d980e0cc721f02c089bbf1"

```

```

    },
    {
      "id": "00467ee724d980e0cc721f02c089bbf1",
      "action": {
        "args": [
          {
            "key": "destination",
            "type": "string",
            "value": "5f4f4f37228e35001204bf77",
            "value_alias": {
              "value": "5f4f4f37228e35001204bf77",
              "alias": "3F_FMS_Military"
            },
            "options": {
              "user_input": false
            },
            "isParamArgs": false
          },
          {
            "key": "goal_finishing",
            "type": "boolean",
            "value": true,
            "value_alias": {
              "alias": "True",
              "value": true
            },
            "options": {
              "user_input": false
            },
            "isParamArgs": false
          },
          {
            "key": "finishing_timeout",
            "type": "number",
            "value": 30,
            "value_alias": {
              "alias": "30",
              "value": 30
            },
            "options": {

```

```

        "max": 300,
        "min": 0,
        "user_input": true
      },
      "isParamArgs": false
    }
  ],
  "func_name": "move"
},
"next": null
}
],
"task": "60375872de1d14e2a97eb0d3",
"rules": [],
"events": []
},
"shared": [],
"created_at": "2021-03-03T06:34:04.589Z",
"updated_at": "2021-03-03T06:43:29.885Z",
"id": "603f2ddcff14a3002a541035"
}
]
},
"msg_uuid": "get_presets",
"msg_ver": "1.0"
}

```

5.12. 프리셋 실행: run_preset(Command)

- 간략히 CMD_RUN_PRESET로 칭합니다.
- Rocon Service는 Concert에 프리 셋의 실행을 요청하는데 이는 프리 셋의 내용으로 새로운 Job생성하는 것과 동일합니다. 따라서 Job생성 요청 결과를 확인하고 사용자에게 Result Message를 전송합니다. 이후 로봇은 Job을 받아서 수행하게 됩니다. 수행 중인 Job의 진행 사항은 사용자가 수신하는 Feedback message의 status, job_id, 로봇의 이동 위치 정보 등을 바탕으로 수행 중임을 알 수 있습니다.
- 정상적인 실행에 성공하면 CMD_COMPOSE_JOB과 동일하게 job_id가 포함된 결과를 회신 받습니다.
- job이 생성되어 실행되므로 CMD_CANCEL_JOB, CMD_PAUSE_JOB, CMD_RESUME_JOB과 같은 job operation 명령을 사용할 수 있습니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"run_preset"	String
params			Object
	preset_id	프리셋 조회로 얻은 리스트에서 선택된 id 를 지정합니다.	String

<표 5-11 run_preset Message 의 msg_body 구성>

run_preset Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "run_preset",
    "params": {
      "preset_id": "603f2ddcff14a3002a541035"
    }
  },
  "msg_uuid": "run_preset",
  "msg_ver": "1.0"
}
```

run_preset 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "run_preset",
    "result": "success",
    "data_contents": {
      "job_id": "603f37f6d23534002aa622c4"
    }
  },
  "msg_uuid": "run_preset",
  "msg_ver": "1.0"
}
```

5.13. 이동: move(Command)

- 간략히 CMD_MOVE로 표시합니다.
- "move"명령은 "navigate", "service"모드에서 사용가능한 명령입니다.

- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다.
- Concert를 통합 Job 생성이 아니라 지정된 좌표 정보로 이동하라는 명령이므로 좌표 정보의 유효성을 사용자가 확인해야 합니다.
- 현재 compose_job에 의해 로봇이 수행 중일 때 navigate 모드의 move 명령이 전달되면 running job 우선에 의해 move 명령은 failure 처리됩니다.
- 로봇이 이동하기 전에 docking여부를 확인하고 docking된 상태라면 자동 undocking을 수행하고 이동합니다.
- 로봇의 현재 위치에서 지정된 pose 정보의 포인트까지 이동하라는 명령입니다.
- 정상적으로 명령 수행 중의 로봇의 위치 정보는 feedback Message에 업데이트되어 전송됩니다.

msg_body 필드명		설명 및 설정 값	자료형
name		"move"	String
params			Object
	pose		Object
	x	맵에서의 이동할 위치의 좌표 x 정보입니다.	Float
	y	맵에서의 이동할 위치의 좌표 y 정보입니다.	Float
	theta	맵에서의 이동할 위치의 좌표 방향 정보입니다. (radian 값)	Float

<표 5-12 move Message의 msg_body 구성>

move Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "move",
    "params": {
      "pose": {
        "x": 14.0345675,
        "y": 32.234784857,
        "theta": -0.34573728484
      }
    }
  },
  "msg_uuid": "move_1",
  "msg_ver": "1.0"
}
```

move 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "move",
    "result": "success"
  },
  "msg_uuid": "move_1",
  "msg_ver": "1.0"
}
```

5.14. 이동 확장: move_ext(Command)

- 간략히 CMD_MOVE_EXT로 표시합니다.
- "move" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다.
- 로봇의 현재 위치에서 연속된 여러개의 지정된 pose 정보와 옵션들로 이루어진 이동하라는 명령입니다.
- 정상적으로 명령 수행 중의 로봇의 위치 정보는 feedback Message에 업데이트되어 전송됩니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"move_ext"	String
params			Object
	waypoints		Array
		pose	Object
		x	맵에서의 이동할 위치의 좌표 x 정보입니다.
		y	맵에서의 이동할 위치의 좌표 y 정보입니다.
		theta	맵에서의 이동할 위치의 좌표 방향 정보입니다. (radian 값)
	seamless		Boolean
	tolerance		Object
		radius	Float
		just_in_goal	Boolean

			result 를 반환하고 멈춘다.	
		timeout	로봇 내부 사유로 이동 진행이 되지 않을 경우의 timeout 값 (기본값 10 초) Just_in_goal 이 false 라면, 해당 timeout 값만큼 기다리고 max_trial 횟수가 초과될 경우 다음 목적지로 이동한다.	Float

<표 513 move_ext Message 의 msg_body 구성>

move_ext Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "move_ext",
    "params": {
      "waypoints": [
        {
          "pose": {
            "x": 0,
            "y": 0,
            "theta": 0
          },
          "seamless": true,
          "tolerance": {
            "radius": 0.5,
            "just_in_goal": false,
            "timeout": 10
          }
        },
        ... 복수개의 waypoint ...
        {
          "pose": {
            "x": 4,
            "y": 0,
            "theta": 1.57
          },
          "seamless": false,
          "tolerance": {
            "radius": 0.5,
            "just_in_goal": false,
            "timeout": 10
          }
        }
      ]
    }
  }
}
```

```

    }
  },
  "msg_uuid": "move_ext_1",
  "msg_ver": "1.0"
}

```

move_ext 결과 Message 예시

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "move_ext",
    "result": "success"
  },
  "msg_uuid": "move_ext_1",
  "msg_ver": "1.0"
}

```

5.15. 멈춤: stop(Command)

- 간략히 CMD_STOP으로 표시합니다.
- "move"명령은 "navigate", "service"모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다.
- 로봇이 현재 단위 명령을 수행하고 있을 때 현재 명령을 취소할 때 사용합니다.
- params 필드가 생략됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"stop"	String

<표 5-13 cancel_job Message 의 msg_body 구성>

stop Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "stop"
  },
}

```

```
{
  "msg_uuid": "stop_1",
  "msg_ver": "1.0"
}
```

stop 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "stop",
    "result": "success"
  },
  "msg_uuid": "stop_1",
  "msg_ver": "1.0"
}
```

5.16. 일시정지: pause(Command)

- 간략히 CMD_PAUSE로 표시합니다.
- "pause"명령은 "navigate", "service"모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다.
- 로봇이 현재 단위 명령을 수행하고 있을 때 현재 명령을 일시 정지할 때 사용합니다.
- params 필드가 생략됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"pause"	String

<표 5-14 pause Message 의 msg_body 구성>

pause Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "pause"
  },
  "msg_uuid": "pause_1",
  "msg_ver": "1.0"
}
```

pause 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "pause",
    "result": "success"
  },
  "msg_uuid": "pause_1",
  "msg_ver": "1.0"
}
```

5.17. 재개: resume(Command)

- 간략히 CMD_RESUME으로 표시합니다.
- "resume" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다.
- 로봇이 현재 단위 명령을 수행 중 일시 정지 상태일 때 해제하고 재개할 때 사용합니다.
- params 필드가 생략됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"resume"	String

<표 5-15 resume Message 의 msg_body 구성>

resume Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "resume"
  },
  "msg_uuid": "resume_1",
  "msg_ver": "1.0"
}
```

resume 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "resume",
    "result": "success"
  }
}
```

```

    },
    "msg_uuid": "resume_1",
    "msg_ver": "1.0"
  }

```

5.18. 충전: dock(Command) - Service API v1.4.0 부터 신규 추가.

- 간략히 CMD_DOCK으로 표시합니다.
- "dock" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다
- 로봇은 현재의 위치에서 주위에 마커 정보를 찾아 존재한다면 docking을 시도합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"dock"	String
params	세부 params 를 설정합니다. 왼쪽(id), 오른쪽(rid) 마커값을 정확히 기입하세요.	Object

<표 5-16 dock Message 의 msg_body 구성>

dock Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "dock",
    "params": {
      "pose": {
        "x": 0,
        "y": 0,
        "theta": 0
      },
      "direction": "forward",
      "barrier_x": -2,
      "chargeable": true,
      "use_docker": true,
      "parking_offset": {
        "x": 0,
        "y": 0,

```

```

    "theta": 0
  },
  "parking_type": "Front",
  "no_markers": 1,
  "markers": [
    {
      "no": 0,
      "id": 100,
      "rid": 4,
      "x": 0,
      "y": 0,
      "z": 0.37,
      "rx": -1.5707963267948966,
      "ry": 0,
      "rz": -1.5707963267948966
    }
  ],
  "charge_switch": true,
  "method": "marker",
  "line": null
}
},
"msg_uuid": "dock_1",
"msg_ver": "1.0"
}

```

dock 결과 Message 예시

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "dock",
    "result": "success",
    "user_auth_info": {
      "user_id": "spt",
      "user_ip": "127.0.0.1",
      "user_name": "blcho",
      "user_token": "RCMUSER_TOKEN_1710470713723"
    }
  }
}

```


5.19. 맵 다운로드: download_map_image(Command)

- 간략히 CMD_DOWNLOAD_MAP_IMAGE로 표시합니다.
- "download_map_image" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 로봇이 현재 위치하고 있는 맵(Grid맵 혹은 모노크롬 맵)의 다운로드를 요청할 때 사용합니다.
- 맵 다운로드 요청 시 정상적인 result 형태의 json message가 서버에서 전송되고 연이어서 file message가 별도로 전송됩니다. file message의 payload 구조는 아래의 "File message의 payload 구조"를 참고하세요.

msg_body 필드 명	설명 및 설정 값	자료형
name	"download_map_image"	String
params		Object
image_type	"gridmap" or "monochrome"(일반적으로 grid 를 사용) "gridmap": FMUI 상에서 사용하는 grid map 이미지 "monochrome": grid 맵을 축소한 모노크롬 타입의 이미지	String

<표 5-17 download_map_image Message 의 msg_body 구성>

download_map_image Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "download_map_image",
    "params": {
      "image_type": "gridmap"
    }
  },
  "msg_uuid": "download_map_image_1",
  "msg_ver": "1.0"
}
```

download_map_image 결과 message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "download_map_image",
    "result": "success",
  }
}
```

```

"error_info": null,
"data_contents_type": "map_info",
"data_contents": {
  "map_id": "5f4f2de9429b4a0012d40c54",
  "map_name": "3F",
  "file_name": "gridmap1599024617564_flipped.png",
  "file_size": 1470152,
  "created_at": "2020-09-02T05:30:17.579Z",
  "metadata": {
    "gridmap": {
      "origin": {
        "x": 2540,
        "y": 545
      },
      "width": 2926,
      "height": 1496,
      "scale_m2px": 32
    },
    "monochrome": {
      "width": 1000,
      "height": 511
    }
  },
  "file_transfer_key": "gridmap1599024617564_flipped.png"
},
"msg_uuid": "result",
"msg_ver": "1.0"
}

```

download_map_image 결과의 data_contents 구성 설명

data_contents 필드명	설명 및 설정 값	자료형
map_id	map id 가 기록됩니다.	String
map_name	map 의 명칭이 기록됩니다.	String
file_name	파일명이 기록됩니다. (gridmap, monochrome 에 따라 다름)	String
file_size	파일 크기가 기록됩니다. (gridmap, monochrome 에 따라 다름)	Integer
created_at	map 정보 등록 일시가 기록됩니다.	String
metadata	map 의 메타 정보가 기록됩니다.	Object

	gridmap		gridmap 이미지 관련 정보가 기록됩니다.	Object	
		origin	gridmap 상에서의 원점 정보가 기록됩니다.	Object	
			x	gridmap 상에서의 원점 x 좌표가 기록됩니다.	Integer
			y	gridmap 상에서의 원점 y 좌표가 기록됩니다.	Integer
		width	gridmap 이미지의 width 가 기록됩니다.	Integer	
		height	gridmap 이미지의 height 가 기록됩니다.	Integer	
		scale_m2px	1 meter 의 크기를 나타내는 pixel 크기가 기록됩니다. Ex) 32 로 기록되면 32 픽셀이 1m 의 길이 의미함.	Integer	
	monochrome		monochrome 이미지 관련 정보가 기록됩니다.	Object	
		width	monochrome 이미지의 width 가 기록됩니다.	Integer	
		height	monochrome 이미지의 height 가 기록됩니다.	Integer	
file_transfer_key			Binary message 로 파일을 전송할 때 헤더에 기록되는 키 값을 나타냅니다. (통상 파일명을 사용하고 있습니다)	String	

<표 5-18 download_map_image Message 의 data_contents 구성 설명>

* File message 의 payload 구조

- File message는 binary 형태의 message입니다.
- 서버에서 download_map_image의 결과 message를 전송하고 이어서 binary형태의 file_message를 전송하는데 이때 사용되는 file_message의 payload 구조를 설명합니다.
- Header와 File data의 형태로 구성됩니다.
- Header는 Pre_fix_keyword ~ File_size 까지의 부분입니다. File data는 File_contents 부분(실제 이미지 파일 데이터)입니다.
- Header 구성 정보

부분	설명	크기
Pre_fix_keyword	문자열 "FILE_DATA"에 해당하는 9 바이트 데이터가 고정으로 존재합니다.	9 바이트 고정
Data_contents_type_size	Data contents type 의 길이가 기록됩니다.	4 바이트 고정
Data_contents_type	Data contents type 이 기록됩니다. 맵 이미지인 경우 "download_map_image"(18 자) 문자열이 기록됩니다.	Data contents type 의 문자열 길이
File_key_size	File message 의 파일 전송 키의 크기를 나타냅니다. 파일 전송 키는 download_map_image 의 결과 메시지의 data_contents.file_transfer_key 의 길이입니다. (현재 키 값은 파일명으로 정의됨) 4 바이트 고정 길이의 정수입니다.	4 바이트 고정
File_key	File_key_size 크기의 파일키가 기록됩니다.	File_key_size 의 정수 크기(가변)의 키 값
Cmd_uuid_size	커맨드 메시지의 msg_uuid 문자열 길이를 나타냅니다.	4 바이트 고정
Cmd_uuid	커맨드 메시지의 msg_uuid 가 기록됩니다.	커맨드 message 의 uuid 의 문자열 갯수
File_size	전송될 파일 데이터의 크기를 나타냅니다. 4 바이트 고정 길이의 정수입니다.	4 바이트 고정
File_contents	전송될 파일의 실제 binary data 입니다. File_size 만큼의 크기를 가집니다.	File_size 크기의 파일 데이터(가변)

download_map_image 의 file_message 예시(아래 예시는 file_message 의 초기 100 바이트를 바이트 문자열로 표시한 내용입니다).

5.20. 맵 정보 요청: get_map_info(Command)

- 간략히 CMD_GET_MAP_INFO로 표시합니다.

- "get_map_info" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 맵 파일의 생성 날짜 및 정보를 요청합니다.
- client가 기동 될 때 맵을 다운로드하는 과정에서 이전에 다운받은 맵이 있다면 먼저 맵 정보를 요청해서 서버상의 맵 create_at 정보가 마지막으로 다운로드 받은 것과 다르다면 갱신된 것으로 간주하고 download_map_image를 호출하는 것을 권고합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_map_info"	String

<표 5-19 get_map_info Message 의 msg_body 구성>

get_map_info Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_map_info"
  },
  "msg_uuid": "get_map_info _1",
  "msg_ver": "1.0"
}
```

get_map_info 결과 message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_map_info",
    "result": "success",
    "error_info": null,
    "data_contents_type": "map_info",
    "data_contents": {
      "map_id": "5f4f2de9429b4a0012d40c54",
      "map_name": "3F",
      "created_at": "2020-09-02T05:30:17.579Z",
      "metadata": {
        "gridmap": {
          "origin": {
            "x": 2540,
            "y": 545
          }
        }
      }
    }
  }
}
```

```

        "width": 2926,
        "height": 1496,
        "scale_m2px": 32
    },
    "monochrome": {
        "width": 1000,
        "height": 511
    }
}
},
"msg_uuid": "get_map_info_1",
"msg_ver": "1.0"
}

```

5.21. 지도 변경 요청: change_map(Command)

- 간략히 CMD_CHANGE_MAP으로 표시합니다.
- "change_map" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 현재 맵에서 지정된 맵으로의 변경을 요청합니다.
- service 모드에서 compose_job으로 생성된 Job을 수행할 때는 자동으로 지도변경을 하므로 사용자가 별도로 지도변경을 하지 않아도 됩니다.
- 비상 상황에서 임의로 사용자가 로봇을 로봇이 마지막으로 인지하고 있는 맵이 아닌 다른 맵(통상 층의 변경 등)의 특정 위치로 이동시킨 이후에 해당 위치에 대한 지도 적용을 강제로 지정하고 싶을 때 이 기능을 사용하면 됩니다.
- 지도 변경을 요청하면 주어진 기준 pose값을 기준으로 자동으로 지도 변경 및 내부적으로 위치 보정을 수행합니다.
- 성공적으로 지도가 변경되면 결과 메시지에 변경된 지도 정보가 반영된 worker_info를 받게 됩니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"change_map"	String
params			Object
	map_id	변경할 맵의 id 를 지정합니다.	String
	pose	변경할 맵에서 로봇이 위치하는 초기 위치 정보를 지정합니다.	Object
	x	변경된 맵에서 로봇이 위치하는 초기 x 좌표를 지정합니다.	Float

	y	변경된 맵에서 로봇이 위치하는 초기 y 좌표를 지정합니다.	Float
	theta	변경된 맵에서 로봇이 위치하는 초기 각도를 지정합니다.	Float

<표 5-20 change_map Message 의 msg_body 구성>

change_map Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "change_map",
    "params": {
      "map_id": "5f4f2db7429b4a0012d40c52",
      "pose": {
        "x": 0.0,
        "y": 0.0,
        "theta": 0.0
      }
    }
  },
  "msg_uuid": "change_map",
  "msg_ver": "1.0"
}
```

change_map 결과 message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "change_map",
    "result": "success",
    "data_contents_type": "change_map_info",
    "data_contents": {
      "worker_info": {
        "type": "robot",
        "name": "real_blcho",
        "uuid": "real_blcho",
        "task_name": "blcho Task",
        "task_description": "blcho Task",
        "status": "idle",
        "status_p": "idle",

```

```

    "status_toggle": [],
    "map": {
      "id": "5f4f2db7429b4a0012d40c52",
      "name": "1F"
    },
    "pose": {
      "x": 0.0,
      "y": 0.0,
      "theta": 0.0
    },
    "velo": {
      "velo_dx": 0.0,
      "velo_dy": 0.0,
      "velo_dz": 0.0
    },
    "battery": {
      "battery_level": 50.0,
      "now_charging": false,
      "charge_source": "docker"
    },
    "brake_released": false,
    "emergency_alarm": false,
    "emergency_button_pressed": false,
    "manual_mode": false,
    "paused": false,
    "in_charging": false,

    "path_plan": null,
    "odometry": {
      "orient_w": 0.0786,
      "orient_x": -0.0,
      "orient_y": -0.0,
      "orient_z": -0.9969,
      "position_x": -0.7723,
      "position_y": -0.3121,
      "position_z": 0.0,
      "velo_dx": 0.0,
      "velo_dy": 0.0,
      "velo_dz": 0.0
    }
  }

```



```

    }
  }
},
"msg_uuid": "change_map",
"msg_ver": "1.0"
}

```

5.22. 위치 보정 요청: relocation(Command)

- 간략히 CMD_RELOCATION으로 표시합니다.
- "relocation" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 현재 맵의 지정된 pose 위치에서 새로운 위치 보정 작업을 요청합니다.
- 로봇은 입력된 위치 좌표를 기준으로 지도에서의 로봇의 위치를 보정하기 위해서 자동 탐색을 합니다. 성공적으로 수행되면 로봇의 위치가 보정됩니다.
- 통상 지도상의 로봇의 위치와 실제 위치가 틀릴 경우, 실제 로봇의 현재 위치 값을 pose값으로 지정하여 보정을 요청하면 됩니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"relocation"	String
params			Object
	pose	현재 맵에서 로봇이 위치하는 위치 정보를 지정합니다.	Object
	x	현재 맵에서 로봇이 위치하는 x 좌표를 지정합니다.	Float
	y	현재 맵에서 로봇이 위치하는 y 좌표를 지정합니다.	Float
	theta	현재 맵에서 로봇이 위치하는 각도를 지정합니다.	Float

<표 5-21 relocation Message 의 msg_body 구성>

relocation Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "relocation",
    "params": {
      "pose": {
        "x": 0.0,
        "y": 0.0,
        "theta": 0.0
      }
    }
  }
}

```

```
{
  "msg_uuid": "relocation",
  "msg_ver": "1.0"
}
```

relocation 결과 message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "relocation",
    "result": "success"
  },
  "msg_uuid": "relocation",
  "msg_ver": "1.0"
}
```

5.23. 장소 인식 요청: set_position(Command)

- 간략히 CMD_SET_POSITION으로 칭합니다.
- 이 명령은 지도 변경과 위치 지정을 하나의 명령으로 지시하는 명령입니다.
- Params에 by_waypoint, by_map_pose 두가지 옵션을 제공합니다. 둘 중 하나의 정보를 지정하면 됩니다.
- by_waypoint는 FMS에 등록해서 사용하고 있는 waypoint의 alias 명칭을 이용해서 장소 지정을 합니다.
 - FMS에 등록된 waypoint의 alias 명칭으로 내부적으로 맵의 id와 wapoint의 pose값을 기준으로 장소 지정을 수행합니다. 간단히 알려진 waypoint의 명칭으로 처리할 수 있는 장점이 있습니다.
 - 로봇을 waypoin에 위치시킬 때 x, y, 좌표 및 theta의 방향도 유사한 방향으로 위치시킬 것을 권고합니다.
 - range_radius는 디폴트 3m로 설정되어 있습니다.
- by_map_pose는 사용자가 지정한 맵의 id와 특정한 pose 값을 기준으로 장소 인식을 수행합니다. 사용자가 지정하는 맵의 특정 pose값을 기준으로 지정된 반경(meter) 범위내에서 위치 인식을 수행합니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"set_position"	String
params			Object
	by_waypoint		Object
	name	Waypoint 의 name 을 지정합니다. (FMS 에 등록된 alias 명칭) (ex, "1F_101")	String

	range_radius	pose 를 기준으로 로봇의 위치를 인식할 반경(meter 단위)	Float
	by_map_pose		Object
	map_id	변경할 맵의 id 를 지정합니다.	String
	range_radius	pose 를 기준으로 로봇의 위치를 인식할 반경(meter 단위)	Float
	pose	변경할 맵에서 로봇이 위치하는 초기 위치 정보를 지정합니다.	Object
	x	변경된 맵에서 로봇이 위치하는 초기 x 좌표를 지정합니다.	Float
	y	변경된 맵에서 로봇이 위치하는 초기 y 좌표를 지정합니다.	Float
	theta	변경된 맵에서 로봇이 위치하는 초기 각도를 지정합니다.	Float

set_position Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "set_position",
    "params": {
      "by_waypoint": {
        "name": "1F_101",
        "range_radius": 3
      }
    }
  },
  "msg_uuid": "set_position_1",
  "msg_ver": "1.0"
}
```

set_position 결과 message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "set_position",
    "result": "success"
  },
  "msg_uuid": "set_position_1",
  "msg_ver": "1.0"
}
```

5.24. 장애물 정보 제거 요청: clear_obstacle(Command)

- 간략히 CMD_CLEAR_OBSTACLE로 표시합니다.

- "clear_obstacle" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 현재 맵에서 로봇이 인식한 장애물 정보를 제거할 때 사용합니다.
- 로봇이 지도상에서 움직일 때 센서로 취득한 장애물 정보를 가지고 있으며 동적으로 갱신됩니다. 이 명령은 취득된 장애물 정보를 강제로 제거할 때 사용합니다.

clear_obstacle Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "clear_obstacle"
  },
  "msg_uuid": "clear_obstacle",
  "msg_ver": "1.0"
}
```

clear_obstacle 결과 message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "clear_obstacle",
    "result": "success"
  },
  "msg_uuid": "clear_obstacle",
  "msg_ver": "1.0"
}
```

5.25. 최근 실패한 job 확인: get_recent_failed_job(Command)

- 간략히 CMD_GET_RECENT_FAILED_JOB으로 표시합니다.
- 최근에 실패한 job이 있는지 확인합니다.
- 중단된 job이 있다면 job_id를 반환합니다.

msg_body 필드명	설명 및 설정 값	자료형
name	"get_recent_failed_job"	String

<표 5.25 get_recent_failed_job Message 의 msg_body 구성>

get_recent_failed_job Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_recent_failed_job"
  },
  "msg_uuid": "get_recent_failed_job_1",
  "msg_ver": "1.0"
}
```

get_recent_failed_job 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_recent_failed_job",
    "result": "success",
    "data_contents": {
      "job_id": "621f012293cbb80035b749a9"
    }
  },
  "msg_uuid": "get_recent_failed_job_1",
  "msg_ver": "1.0"
}
```

5.26. 최근 실패한 job 실행: run_recent_failed_job(Command)

- 간략히 CMD_RUN_RECENT_FAILED_JOB으로 표시합니다.
- 최근에 실패한 job이 있다면 중단된 지점부터 새로운 job을 생성합니다.
- 실패한 job은 CMD_GET_RECENT_FAILED_JOB에서 job_id로 확인할 수 있습니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"run_recent_failed_job"	String

<표 5.26 run_recent_failed_job Message 의 msg_body 구성>

run_recent_failed_job Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "run_recent_failed_job"
  },
  "msg_uuid": "run_recent_failed_job_1",
  "msg_ver": "1.0"
}
```

run_recent_failed_job 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "run_recent_failed_job",

```

```
{
  "result": "success",
  "data_contents": {
    "job_id": "621f01d193cbb80035b749ab"
  }
},
"msg_uuid": "run_recent_failed_job_1",
"msg_ver": "1.0"
}
```

5.27. 사용자 지정 이벤트 해제: notify_user_event(Command)

- 간략히 CMD_NOTIFY_USER_EVENT로 표시합니다.
- CMD_NOTIFY_USER_EVENT "navigate", "service"모드에서 사용가능한 명령입니다.
- 이벤트를 기다리는 로봇을 풀어주는 명령입니다.
- Event key 값은 get_user_event나 event feedback을 통해 확인할 수 있습니다

msg_body 필드 명	설명 및 설정 값	자료형
key	"event 를 나타내는 key 값"	integer

<표 5.26 notify_user_event Message 의 msg_body 구성>

notify_user_event Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "notify_user_event",
    "params": {
      "event_key": 1648704311000
    }
  },
  "msg_uuid": "notify_user_event_1",
  "msg_ver": "1.0"
}
```

notify_user_event 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "notify_user_event",
    "result": "success",
    "data_contents": {
      "result": true,
      "key": 1648704311000
    }
  },
  "msg_uuid": "notify_user_event_1",
  "msg_ver": "1.0"
}
```

5.28. 사용자 지정 이벤트 보기: get_user_event(Command)

- 간략히 CMD_GET_USER_EVENT 로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 로봇이 기다리고 있는 이벤트들을 보는 명령입니다.

get_user_event Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_user_event"
  },
  "msg_uuid": "get_user_event_1",
  "msg_ver": "1.0"
}
```

get_user_event 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_user_event",
    "result": "success",
    "data_contents": {
      "events": [
        {
          "key": 1648704311000,
          "event_host": "client_service_api",
          "event_type": "wait",
          "start_date": "2022-03-31T14:25:11",
          "timeout": -1,
          "status": "listening"
        }
      ]
    }
  },
  "msg_uuid": "get_user_event_1",
  "msg_ver": "1.0"
}
```

5.29. GPIO의 Input/Output값 확인: get_gpio(Command)

- 간략히 CMD_GET_GPIO로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 로봇의 Digital, Analog의 input/output 값을 배열의 형태로 보여줍니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_gpio"	string

get_gpio Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_gpio"
  },
  "msg_uuid": "get_gpio_1",
  "msg_ver": "1.0"
}
```

get_gpio 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_gpio",
    "result": "success",
    "data_contents_type": "gpio",
    "data_contents": {
      "gpio_data": {
        "d_in_len": 8,
        "d_in": [1, 0, 1, 0, 0, 0, 0, 0],
        "d_out_len": 8,
        "d_out": [1, 0, 0, 0, 0, 0, 0, 0]
      },
      "a_in_len": 8,
      "a_in": [1, 0, 0, 0, 0, 0, 0, 0],
      "a_out_len": 8,
      "a_out": [1, 0, 0, 1, 0, 0, 0, 0]
    }
  },
  "msg_uuid": "get_gpio_1",
  "msg_ver": "1.0"
}
```

5.30. GPIO의 digital output값 설정: set_d_out(Command)

- 간략히 CMD_SET_D_OUT으로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 변경하길 원하는 digital output channel 번호와 변경하고자 하는 값(0,1)을 설정합니다.

msg_body 필드 명	설명 및 설정 값	자료형
---------------	-----------	-----

name	"set_d_out"	String
params		Object
d_out_bit	d_out_bit key 는 digital output 의 변경하길 원하는 값을 채널 순서대로 내부 0 또는 1 인 array 로 value 를 설정합니다.	Array
mask_bit	mask_bit key 는 변경하길 원하는 channel 을 선택하는 내부 0 또는 1 인 bit 로 설정합니다.	Array

set_d_out Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "set_d_out",
    "params": {
      "d_out_bit": [1, 0, 0, 1, 1, 1, 1, 1],
      "mask_bit": [1, 0, 1, 1, 1, 1, 1, 1]
    }
  },
  "msg_uuid": "set_d_out_1",
  "msg_ver": "1.0"
}
```

set_d_out 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "set_d_out",
    "result": "success"
  },
  "msg_uuid": "set_d_out_1",
  "msg_ver": "1.0"
}
```

5.31. GPIO의 analog output값 설정: set_a_out(Command)

- 간략히 CMD_SET_A_OUT으로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 변경하길 원하는 analog output channel 번호와 변경하고자 하는 값(0,1)을 설정합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"set_a_out"	String
params		Object
a_out_bit	a_out_bit key 는 analog output 의 변경하길 원하는 값을 채널 순서대로 내부 0 또는 1 인 array 로 value 를 설정합니다.	Array

mask_bit	mask_bit key 는 변경하길 원하는 channel 을 선택하는 내부 0 또는 1 인 bit 로 설정합니다.	Array
----------	---	-------

set_a_out Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "set_a_out",
    "params": {
      "a_out_bit": [1, 0, 0, 1, 1, 1, 1, 1],
      "mask_bit": [1, 0, 1, 1, 1, 1, 1, 1]
    }
  },
  "msg_uuid": "set_a_out_1",
  "msg_ver": "1.0"
}
```

set_a_out 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "set_a_out",
    "result": "success"
  },
  "msg_uuid": "set_a_out_1",
  "msg_ver": "1.0"
}
```

5.32. GPIO의 digital input값 등록: get_d_in(Command)

- 간략히 CMD_GET_D_IN으로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 등록하길 원하는 digital input 채널번호와 변경하고자 하는 값(0,1)을 설정합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_d_in"	String
params		Object
d_in_bit	d_in_bit key 는 digital input 의 등록하길 원하는 값을 채널 순서대로 내부 0 또는 1 인 배열로 설정합니다.	Array
mask_bit	mask_bit key 는 등록하길 원하는 channel 을 내부 0 또는 1 인 배열로 설정합니다.	Array

get_d_in Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_d_in",
    "params": {
      "d_in_bit": [1, 1, 1, 1, 1, 1, 1, 1],
      "mask_bit": [0, 0, 0, 0, 0, 0, 1, 1]
    }
  },
  "msg_uuid": "get_d_in_1",
  "msg_ver": "1.0"
}
```

get_d_in 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_d_in",
    "result": "success"
  },
  "msg_uuid": "get_d_in_1",
  "msg_ver": "1.0"
}
```

5.33. GPIO의 analog input값 등록: get_a_in(Command)

- 간략히 CMD_GET_A_IN으로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 등록하길 원하는 Analog input 채널번호와 변경하고자 하는 값(0,1)을 설정합니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"get_a_in"	String
params			Object
	a_in_bit	a_in_bit key 는 analog input 의 등록하길 원하는 값을 채널 순서대로 내부 0 또는 1 인 배열로 설정합니다.	Array
	mask_bit	mask_bit key 는 등록하길 원하는 channel 을 내부 0 또는 1 인 배열로 설정합니다.	Array

get_a_in Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_a_in",
    "params": {
      "a_in_bit": [1, 1, 1, 1, 1, 1, 1, 1],

```

```

    "mask_bit": [0, 0, 0, 0, 0, 0, 1, 1]
  }
},
"msg_uuid": "get_a_in_1",
"msg_ver": "1.0"
}

```

get_a_in 결과 Message 예시

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "get_a_in",
    "result": "success"
  },
  "msg_uuid": "get_a_in_1",
  "msg_ver": "1.0"
}

```

5.34. GPIO의 Digital input 입력 대기: d_in_listener(Command)

- 간략히 CMD_D_IN_LISTENER로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 입력되는 값을 받길 원하는 digital input 값과 얻고자 하는 채널을 설정합니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"d_in_listener"	String
params			Object
	d_in_bit	d_in_bit key 는 digital input 의 입력 받길 원하는 값을 채널 순서대로 내부 0 또는 1 인 배열로 설정합니다.	Array
	mask_bit	mask_bit key 는 등록하길 원하는 channel 을 내부 0 또는 1 인 배열로 설정합니다.	Array

d_in_listener Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "d_in_listener",
    "params": {
      "d_in_bit": [1, 1, 1, 1, 1, 1, 1, 1],
      "mask_bit": [0, 0, 0, 0, 0, 0, 1, 1]
    }
  },
  "msg_uuid": "d_in_listener_1",
  "msg_ver": "1.0"
}

```

d_in_listener 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "d_in_listener",
    "result": "success"
  },
  "msg_uuid": "d_in_listener_1",
  "msg_ver": "1.0"
}
```

5.35. GPIO의 Analog input 입력 대기: a_in_listener(Command)

- 간략히 CMD_A_IN_LISTENER로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 입력되는 값을 받길 원하는 analog input 값과 얻고자 하는 채널을 설정합니다.
- a_in_bit key는 analog input의 변경 받길 원하는 값을 채널 순서대로 내부 0또는1인 array로 value를 설정하고, mask_bit key는 변경 받길 원하는 채널을 선택하는 내부 0또는1의 array로 값을 설정합니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"a_in_listener"	String
params			Object
	a_in_bit	a_in_bit key 는 analog input 의 입력 받길 원하는 값을 채널 순서대로 내부 0 또는 1 인 배열로 설정합니다.	Array
	mask_bit	mask_bit key 는 등록하길 원하는 channel 을 내부 0 또는 1 인 배열로 설정합니다.	Array

a_in_listener Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "a_in_listener",
    "params": {
      "a_in_bit": [1, 1, 1, 1, 1, 1, 1, 1],
      "mask_bit": [0, 0, 0, 0, 0, 0, 1, 1]
    }
  },
  "msg_uuid": "a_in_listener_1",
  "msg_ver": "1.0"
}
```

a_in_listener 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "a_in_listener",
    "result": "success"
  },
  "msg_uuid": " a_in_listener_1",
  "msg_ver": "1.0"
}
```

5.36. Suspend 해제: resume_for_suspend(Command)

- 간략히 CMD_RESUME_FOR_SUSPEND로 표시합니다.
- "navigate", "service"모드에서 사용가능한 명령입니다.
- 로봇이 Suspend 상태에 있을 때 사용자 확인에 의해 해제할 때 사용하는 명령입니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"resume_for_suspend"	String

Resume_for_suspend Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "resume_for_suspend"
  },
  "msg_uuid": "resume_for_suspend_1",
  "msg_ver": "1.0"
}
```

Resume_for_suspend 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "resume_for_suspend",
    "result": "success",
  },
  "msg_uuid": "resume_for_suspend_1",
  "msg_ver": "1.0"
}
```

}

5.37. Warning Notice 열기: get_warning_notice(Command)

- 간략히 CMD_GET_WARNING_NOTICE로 표시합니다.
- "service"모드에서 사용가능한 명령입니다.
- Rocon Client가 warning 상태일 때의 warning 정보를 얻을 수 있습니다.
- 현재 발생된 경고 고지는 배열 형태의 결과로 반환됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_warning_notice"	String

Get_warning_notice Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_warning_notice"
  },
  "msg_uuid": "get_warning_notice_1",
  "msg_ver": "1.0"
}
```

Get_warning_notice 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_warning_notice",
    "result": "success",
    "data_contents_type": "warning_notice",
    "data_contents": {
      "enabled": true,
      "last_updated_ms": null,
      "warnings": []
    }
  },
  "msg_uuid": "get_warning_notice_1",
  "msg_ver": "1.0"
}
```

5.38. Warning Notice 설정: set_warning_notice(Command)

- 간략히 CMD_SET_WARNING_NOTICE로 표시합니다.
- "service"모드에서 사용가능한 명령입니다.
- Rocon Client에 강제 경고 고지를 설정하는 데 사용됩니다. 경고 고지가 더 이상 유효하지 않은 경우 사용자가 지워야 합니다.
- CMD_SET_WARNING_NOTICE로 입력된 에러 고지의 카테고리는 자동으로 "user"로 설정됩니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"set_warning_notice"	String
params			Object
	category	카테고리를 기입합니다. (Rocon Client 예약어 : "fms", "rcs", "robot").	String
	warning_info		Object
	code	경고코드를 기입합니다. (필수).	String
	level	경고 레벨을 지정합니다. (필수 : "warning")	String
	module	경고에 대한 세부 모듈명을 기입합니다. (선택)	String
	description	경고에 대한 설명을 기입합니다. (필수)	String
	recovery_info	경고 복구 방법을 기입합니다. (선택)	String

Set_warning_notice Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "set_warning_notice",
    "params": {
      "category": "user",
      "warning_info": {
        "code": "USER_W0001",
        "level": "warning",
        "module": "user_app",
        "description": "This is a test warning"
      },
      "recovery_info": null
    }
  },
  "msg_uuid": "set_warning_notice_1",
```



```
"msg_ver": "1.0"
}
```

Set_warning_notice 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "set_warning_notice",
    "result": "success"
  },
  "msg_uuid": "set_warning_notice_1",
  "msg_ver": "1.0"
}
```

5.39. Warning Notice 지우기: clear_warning_notice(Command)

- 간략히 CMD_CLEAR_WARNING_NOTICE로 표시합니다.
- "service"모드에서 사용가능한 명령입니다.
- code 혹은 category 둘 중 하나를 설정합니다.
- 현재 모든 경고고지를 삭제할 때는 code값으로 "all"을 지정합니다.
- 카테고리를 지정하면 해당 카테고리의 모든 경고고지를 해지합니다.
- 특정 경고코드를 지정하면 해당 에러고지만 해지합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"clear_warning_notice"	String
params		Object
code	"all" 또는 "특정 경고코드"를 지정합니다.	String
category	특정 카테고리 지정합니다.	String

Clear_warning_notice Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "clear_warning_notice",
    "params": {
      "code": "all"
    }
  },
  "msg_uuid": "clear_warning_notice_1",
}
```

```
"msg_ver": "1.0"
}
```

Clear_warning_notice 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "clear_warning_notice",
    "result": "success"
  },
  "msg_uuid": "clear_warning_notice_1",
  "msg_ver": "1.0"
}
```

5.40. Error Notice 얻기: get_error_notice(Command)

- 간략히 CMD_GET_ERROR_NOTICE로 표시합니다.
- "service"모드에서 사용가능한 명령입니다.
- Rocon Client가 error 상태일 때의 error 정보를 얻을 수 있습니다.
- 현재 발생된 에러 고지는 배열 형태의 결과로 반환됩니다.
- params 필드가 생략됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_error_notice"	String

Get_error_notice Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_error_notice"
  },
  "msg_uuid": "get_error_notice_1",
  "msg_ver": "1.0"
}
```

Get_error_notice 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_error_notice",
```

```

"result": "success",
"data_contents_type": "error_notice",
"data_contents": {
  "enabled": true,
  "last_updated_ms": 1660094997537,
  "errors": [
    {
      "category": "user",
      "error_info": {
        "code": "USER_E001",
        "level": "error",
        "module": "user_app",
        "description": "this is a test error"
      },
      "first_date": "2022-08-10T10:29:57",
      "last_date": "2022-08-10T10:29:57",
      "repeat_count": 1
    }
  ]
},
"msg_uuid": "get_error_notice_1",
"msg_ver": "1.0"
}

```

5.41. Error Notice 설정: set_error_notice(Command)

- 간략히 CMD_SET_ERROR_NOTICE로 표시합니다.
- "service"모드에서 사용가능한 명령입니다.
- Rocon Client가 error 상태일 때의 error 정보를 얻을 수 있습니다.
- 현재 발생된 에러 고지는 배열 형태의 결과로 반환됩니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"set_error_notice"	String
params			Object
	category	카테고리를 기입합니다. (Rocon Client 예약어 : "fms", "rcs", "robot").	String
	error_info		Object
	code	에러코드를 기입합니다. (필수).	String
	level	에러 레벨을 지정합니다. (필수 : "error" or "critical")	String

	module	에러에 대한 세부 모듈명을 기입합니다. (선택)	String
	description	에러에 대한 설명을 기입합니다. (필수)	String
	recovery_info	에러 복구 방법을 기입합니다. (선택)	String

Set_error_notice Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "get_error_notice"
  },
  "msg_uuid": "get_error_notice_1",
  "msg_ver": "1.0"
}
```

Set_error_notice 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_error_notice",
    "result": "success",
    "data_contents_type": "error_notice",
    "data_contents": {
      "enabled": true,
      "last_updated_ms": 1660094997537,
      "errors": [
        {
          "category": "user",
          "error_info": {
            "code": "USER_E001",
            "level": "error",
            "module": "user_app",
            "description": "this is a test error"
          },
          "first_date": "2022-08-10T10:29:57",
          "last_date": "2022-08-10T10:29:57",
          "repeat_count": 1
        }
      ]
    }
  },
}
```

```

"msg_uuid": "get_error_notice_1",
"msg_ver": "1.0"
}

```

5.42. Error Notice 지우기: clear_error_notice(Command)

- 간략히 CMD_CLEAR_ERROR_NOTICE로 표시합니다.
- "service"모드에서 사용가능한 명령입니다.
- code 혹은 category 둘 중 하나를 설정합니다.
- 현재 모든 에러고지를 삭제할 때는 code값으로 "all"을 지정합니다.
- 카테고리를 지정하면 해당 카테고리의 모든 에러고지를 해지합니다.
- 특정 에러코드를 지정하면 해당 에러고지만 해지합니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"clear_error_notice"	String
params			Object
	code	"all" 또는 "특정 에러코드"를 지정합니다.	String
	category	특정 카테고리 지정합니다.	String

Clear_error_notice Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "clear_error_notice",
    "params": {
      "code": "all"
    }
  },
  "msg_uuid": "clear_error_notice_1",
  "msg_ver": "1.0"
}

```

Clear_error_notice 결과 Message 예시

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "clear_error_notice",
    "result": "success"
  }
}

```

```
{
  "msg_uuid": "clear_error_notice_1",
  "msg_ver": "1.0"
}
```

5.43. 동적 속성(Dynamic Property) 설정: set_property(Command)

- 간략히 CMD_SET_PROPERTY로 표시합니다.
- "service"모드에서 사용가능한 명령입니다.
- Params 내부 mapper, motion, lift 파라미터는 모두 optional parameter이며, 선택적 사용 혹은 조합 사용이 가능합니다.
- mapper에는 로봇이 진입할 지형을 landform key에 value로써 입력합니다.
- landform은 현재 'slope', 'tunnel', 'flat'(기본값) | 사용 가능합니다.
- motion에는 로봇의 최대속도 및 최대 각속도를 설정합니다. lift에는 로봇이 lift up/down시 대차의 크기에 따라 변경되는 풋프린트 사이즈를 입력합니다.

msg_body 필드 명		설명 및 설정 값	자료형
name		"set_property"	String
params			Object
	mapper		Object
	landform	'flat' 'slope' 'tunnel'	String
	motion		Object
	max_speed	로봇의 max_speed 스펙 내부의 값으로 설정합니다.	Float
	max_angular_speed	로봇의 max_angular_speed 스펙 내부의 값으로 설정합니다.	Float
	lift		Object
	load	대차를 들어 올리고 있을지에 대한 여부를 설정	Boolean
	load_size_x	대차의 width 에 해당하는 값을 설정	Float
	load_size_front_to_center	대차의 중심에서 전면부까지에 해당하는 length 값을 설정	Float
	load_size_back_to_center	대차의 중심에서 후면부까지에 해당하는 length 값을 설정	Float

Set_property Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
```

```

"name": "set_property",
"params": {
  "mapper": {
    "landform": "flat"
  },
  "motion": {
    "max_speed": 1,
    "max_angular_speed": 0.6
  },
  "lift": {
    "load": true,
    "load_size_x": 0.965,
    "load_size_front_to_center": 0.7,
    "load_size_back_to_center": 0.7
  }
},
"msg_uuid": "set_property_1",
"msg_ver": "1.0"
}

```

Set_property 결과 메시지 예시

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "set_property",
    "result": "success",
    "data_contents_type": "dynamic_property"
  },
  "msg_uuid": "set_property_1",
  "msg_ver": "1.0"
}

```

5.44. 동적 속성(Dynamic Property) 불러오기 : get_property(Command)

- 간략히 CMD_GET_PROPERTY로 표시합니다.

- "service"모드에서 사용가능한 명령입니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"get_property"	String

get_property 결과 메시지 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "get_property",
    "result": "success",
    "data_contents_type": "dynamic_property",
    "data_contents": {
      "contents": {
        "properties": {
          "lift": {
            "load": true,
            "load_size_x": 0.965,
            "load_size_front_to_center": 0.7,
            "load_size_back_to_center": 0.7
          },
          "mapper": {
            "landform": "flat"
          },
          "motion": {
            "max_angular_speed": 0.6,
            "max_speed": 1
          }
        }
      },
      "msg_type": "data_result",
      "result": "success",
      "uuid": "req_get_property_65"
    }
  },
  "msg_uuid": "get_property_1",
  "msg_ver": "1.0"
}
```


5.45. 충전: charge(Command) - Service API v1.2.6 부터 지원하지 않습니다. (CMD_DOCK으로 대체됨)

- 간략히 CMD_CHARGE로 표시합니다.
- "charge"명령은 "navigate", "service"모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다
- 로봇은 현재의 위치에서 주위에 마커 정보를 찾아 존재한다면 docking을 시도합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"charge"	String
params	세부 params 를 설정합니다. 왼쪽(id), 오른쪽(rid) 마커값을 정확히 기입하세요.	Object

Charge Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "charge",
    "params": {
      "pose": {
        "x": 0,
        "y": 0,
        "theta": 0
      },
      "direction": "forward",
      "barrier_x": -2,
      "chargeable": true,
      "use_docker": true,
      "parking_offset": {
        "x": 0,
        "y": 0,
        "theta": 0
      },
      "parking_type": "Front",
      "no_markers": 1,
      "markers": [
        {
```

```

        "no": 0,
        "id": 100,
        "rid": 4,
        "x": 0,
        "y": 0,
        "z": 0.37,
        "rx": -1.5707963267948966,
        "ry": 0,
        "rz": -1.5707963267948966
    }
]
}
},
"msg_uuid": "charge_1",
"msg_ver": "1.0"
}

```

Charge 결과 Message 예시

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "charge",
    "result": "success"
  },
  "msg_uuid": "charge_1",
  "msg_ver": "1.0"
}

```

5.46. 주차: park(Command)

- 간략히 CMD_PARK 로 표시합니다.
- "park" 명령은 "navigate", "service" 모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다
- 로봇은 현재의 위치에서 주위에 마커 정보를 찾아 존재한다면 정밀 주차를 시도합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"park"	String
params	세부 params 를 설정합니다. 왼쪽(id), 오른쪽(rid) 마커값을 정확히 기입하세요. 마커와 로봇 전면 사이의 거리(미터)를	Object

입력하세요.	
--------	--

park Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "park",
    "params": {
      "pose": {
        "x": 0,
        "y": 0,
        "theta": 0
      },
      "direction": "forward",
      "barrier_x": -2,
      "chargeable": false,
      "use_docker": false,
      "parking_offset": {
        "x": -0.1,
        "y": 0,
        "theta": 0
      },
      "parking_type": "Front",
      "no_markers": 1,
      "markers": [
        {
          "no": 0,
          "id": 100,
          "rid": 4,
          "x": 0,
          "y": 0,
          "z": 0.37,
          "rx": -1.5707963267948966,
          "ry": 0,
          "rz": -1.5707963267948966
        }
      ]
    }
  }
},
```

```
{
  "msg_uuid": "park_1",
  "msg_ver": "1.0"
}
```

park 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "park",
    "result": "success"
  },
  "msg_uuid": "park_1",
  "msg_ver": "1.0"
}
```

5.47. 주차해제: unpark(Command)

- 간략히 CMD_UNPARK로 표시합니다.
- "unpark"명령은 "navigate", "service"모드에서 사용가능한 명령입니다.
- 엔지니어가 로봇의 단위 테스트용으로 사용하는 명령입니다
- CMD_DOCK, CMD_PARK로 수행 후 로봇이 마커로 부터 떨어져 나오기 위해 사용합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"charge"	String
params		Object
direction	언파킹 할 방향을 설정합니다. ("forward" 혹은 "backward")	String
exit distance	언파킹 할 거리를 설정합니다. (미터)	Float

Unpark Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "unpark",
    "params": {
      "direction": "backward",
      "exit_distance": 0.6
    }
  },
  "msg_uuid": "unpark_1",
}
```

```
"msg_ver": "1.0"  
}
```

unpark 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "unpark",
    "result": "success"
  },
  "msg_uuid": "unpark_1",
  "msg_ver": "1.0"
}
```

5.48. 가상 조이스틱 사용 시작: virtual_joystick_start(Command)

- 간략히 CMD_VJ_START로 표시합니다.
- 현재는 navigage 모드에서만 가상 조이스틱 이용 가능합니다. (explore 모드는 향후 지원예정)
- 가장 조이스틱을 사용중에 물리적인 조이스틱을 동시에 사용하지 못합니다. 가상 조이스틱 stop 이후 물리조이스틱을 사용할 수 있습니다.
- 가상 조이스틱 사용 시작을 위해 호출해야하는 명령입니다.
- 가상 조이스틱은 동시에 다중 사용자의 사용을 허락하지 않습니다. 마지막에 CMD_VJ_START를 성공한 사용자가 사용자가 제어권을 가집니다.
- 이미 다른 사용자에게 의해 CMD_VJ_START한 이후 CMD_VJ_FINISH하지않은 상태라면 결과가 "cancelled"로 리턴됩니다.
- CMD_VJ_START 명령 성공시 virtual_joystick_token이 결과에 포함됩니다. 이후, CMD_VJ_EVENT, CMD_VJ_HEALTH_CHECK, CMD_VJ_FINISH 등 모든 가상 조이스틱 명령에 토큰 정보를 사용해야합니다.
- CMD_VJ_START 이후 CMD_VJ_HEALTH_CHECK를 통해서 주기적으로 가상 조이스틱 사용 중임을 체크해야 합니다. 그렇지 않으면 타임아웃으로 가상조이스틱 사용지 강제로 종료됩니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"virtual_joystick_start"	String
params		Object
force_restart	사용중인 가상 조이스틱의 제어권을 강제로 해제하고 다시 시작하기 위해서는 force_restart 플래그를 True 로 설정합니다. (디폴트 false)	Boolean

virtual_joystick_start Message 예시

```
{
  "msg_type": "command",
```

```
{
  "msg_body": {
    "name": "virtual_joystick_start",
    "params": {
      "force_restart": false
    }
  },
  "msg_uuid": "virtual_joystick_start_1"
}
```

virtual_joystick_start 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "virtual_joystick_start",
    "result": "success",
    "data_contents": {
      "virtual_joystick_token": "vj_token_2023-07-04T01:16:31"
    }
  },
  "msg_uuid": "virtual_joystick_start_1",
  "msg_ver": "1.0"
}
```

5.49. 가상 조이스틱 사용 중단: virtual_joystick_finish(Command)

- 간략히 CMD_VJ_FINISH으로 표시합니다.
- 가상 조이스틱 사용을 명시적으로 종료할 때 사용하는 명령입니다.
- CMD_VJ_START 명령 성공시 받은 토큰을 사용하여야 정상적인 종료를 할 수 있습니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"virtual_joystick_finish"	String
params		Object
virtual_joystick_token	CMD_VJ_START 명령 성공시 받은 토큰을 기록합니다.	Boolean

virtual_joystick_finish Message 예시

```
{
  "msg_type": "command",
  "msg_body": {
    "name": "virtual_joystick_finish",
    "params": {
```

```

    "virtual_joystick_token": "vj_token_2023-07-03T11:02:40"
  }
},
"msg_uuid": "virtual_joystick_finish_1"
}

```

virtual_joystick_finish 결과 Message 예시

```

{
  "msg_type": "result",
  "msg_body": {
    "name": "virtual_joystick_finish",
    "result": "success"
  },
  "msg_uuid": "virtual_joystick_finish_1",
  "msg_ver": "1.0"
}

```

5.50. 가상 조이스틱 헬스체크: virtual_joystick_health_check(Command)

- 간략히 CMD_VJ_HEALTH_CHECK로 표시합니다.
- CMD_VJ_START 이후 CMD_VJ_HEALTH_CHECK를 통해서 주기적으로 가상 조이스틱 사용 중임을 체크해야 합니다. 그렇지 않으면 타임아웃으로 가상조이스틱 사용지 강제로 종료됩니다.
- 5초에 한번씩 CMD_VJ_HEALTH_CHECK를 RC에 보내어 성공 결과를 확인 받도록 권고합니다. RC는 마지막 체크 메시지를 수신한 이후 최대 30초 이상 수신 메시지가 없으면 강제로 가상 조이스틱 연결을 종료합니다.
- CMD_VJ_HEALTH_CHECK 결과가 failure인 경우 사용자의 가상 조이스틱 사용을 정리하고 다시 조이스틱 사용을 원한다면 CMD_VJ_START 부터 다시 진행합니다.

msg_body 필드 명	설명 및 설정 값	자료형
name	"virtual_joystick_health_check"	String
params		Object
virtual_joystick_token	CMD_VJ_START 명령 성공시 받은 토큰을 기록합니다.	Boolean

virtual_joystick_health_check Message 예시

```

{
  "msg_type": "command",
  "msg_body": {
    "name": "virtual_joystick_health_check",
    "params": {
      "virtual_joystick_token": "vj_token_2023-07-04T01:39:21"
    }
  }
}

```



```
{
  "msg_uuid": "virtual_joystick_health_check_1"
}
```

virtual_joystick_health_check 결과 Message 예시

```
{
  "msg_type": "result",
  "msg_body": {
    "name": "virtual_joystick_health_check",
    "result": "success"
  },
  "msg_uuid": "virtual_joystick_health_check_1",
  "msg_ver": "1.0"
}
```

5.51. 가상 조이스틱 이벤트: virtual_joystick_event(Command)

- 간략히 CMD_VJ_EVENT로 표시합니다.
- 사용자가 조이스틱 핸들을 움직일 때 지정된 3가지 이벤트를 발생시켜야합니다.
- 로봇의 이동을 위해 지정된 범위내의 값을 설정해야합니다.
- "now_dragging" 이벤트는 100ms 주기로 발생시키는 것을 권고합니다.
- CMD_VJ_EVENT는 결과를 반환하지 않습니다. 단방향 전용 메시지입니다

msg_body 필드 명	설명 및 설정 값	자료형
name	"virtual_joystick_event"	String
params		Object
virtual_joystick_token	CMD_VJ_START 명령 성공시 받은 토큰을 기록합니다.	Boolean
event	사용자가 가상 조이스틱 핸들을 움직일 경우 "start_dragging", "now_dragging", "finish_dragging" 세가지의 이벤트를 명시적으로 지정해야합니다.	String
axis_0	가상 조이스틱의 x 축 방향의 움직임을 정수값으로 지정해야합니다. 로봇의 오른쪽 방향으로의 이동은 양수(0 ~ 32767), 왼쪽 방향으로의 이동은 음수(-32767 ~ 0)의 범위로 설정되어야 합니다.	Integer
axis_1	가상 조이스틱의 y 축 방향의 움직임을 정수값으로 지정해야합니다. 로봇의 전진 방향으로의 이동은 음수(-32767 ~ 0), 후진방향의 이동은 양수(0 ~ 32767)의 범위로 설정되어야 합니다.	Integer

max_speed	가상 조이스틱으로 로봇의 주행 최대 속도를 제한합니다. (디폴트 0.6, 범위 0.1 ~ 1.0 m/s)	Float
-----------	---	-------

virtual_joystick_event Message 예시

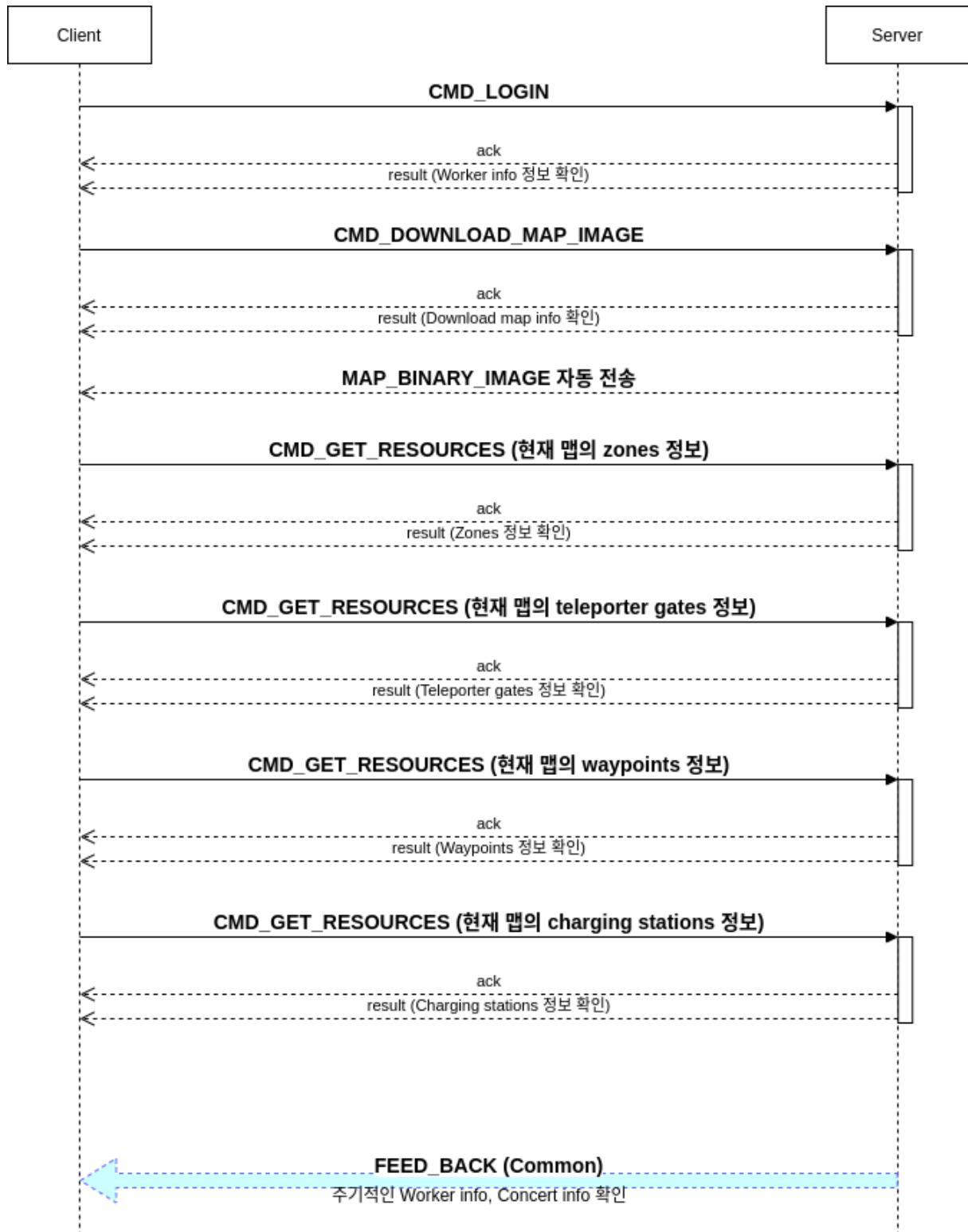
```
{
  "msg_type": "command",
  "msg_body": {
    "name": "virtual_joystick_event",
    "params": {
      "virtual_joystick_token": "",
      "event": "start_dragging",
      "axis_0": 0,
      "axis_1": 0,
      "max_speed": 0.6
    }
  },
  "msg_uuid": "virtual_joystick_event_1"
}
```

virtual_joystick_event 결과 Message 예시

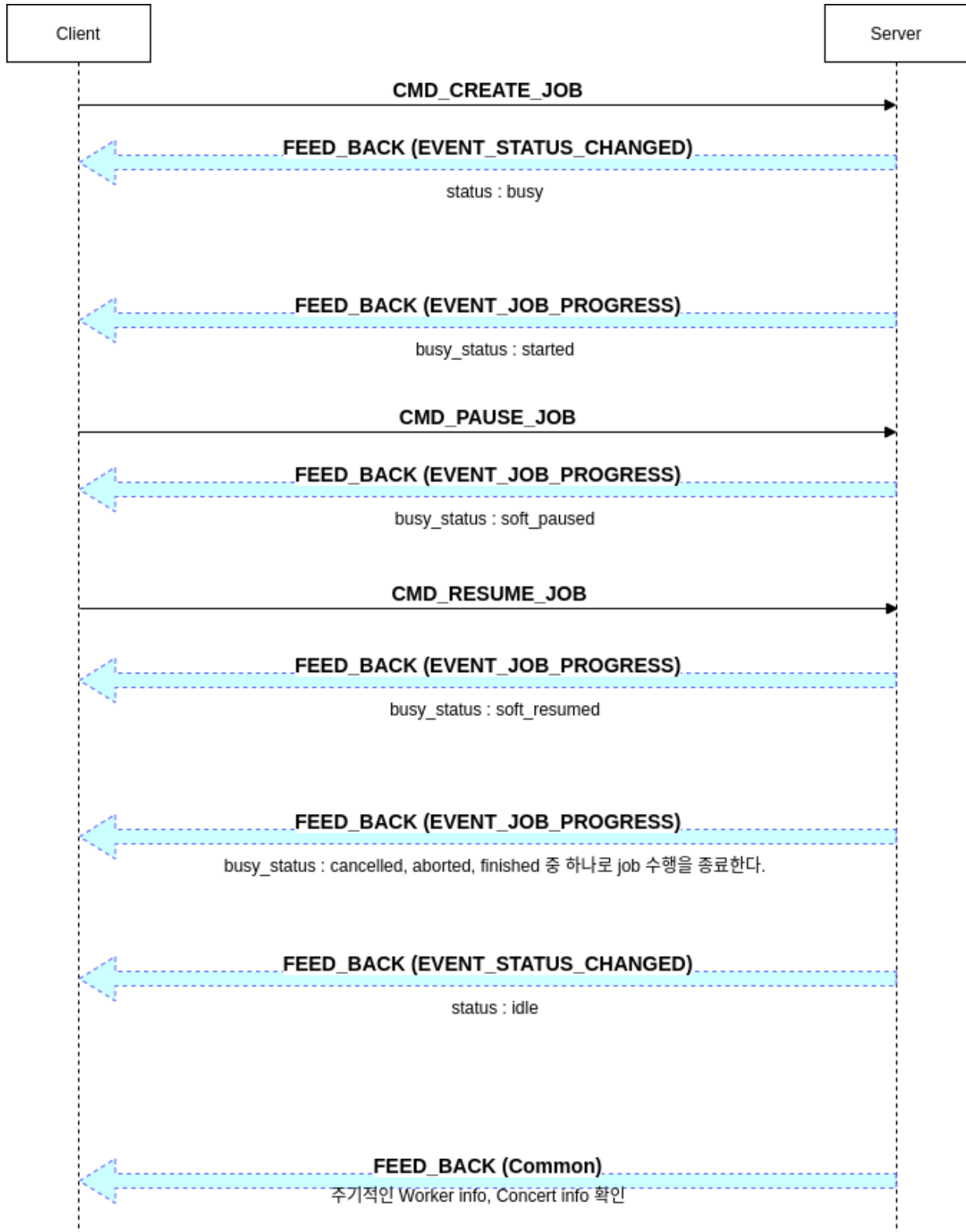
결과 메시지는 없습니다

6. 주요 시나리오별 워크플로우

6.1. 기본 초기화 과정



6.2. Job 수행 시 feedback event 전달 과정



6.3. job수행시 세부 feedback event 예시1

- 1층 waypoint(1_st_hall) 1층 waypoint(1_st_hall)에서 3층 waypoint(3F_elv_front)간 이동

설명	Feedback event
1. Job 시작에 대한 worker 의 상태 변경	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_status_changed", ...(생략) "event_info": { "result": "success", "data": { "prev_status_p": "idle", "prev_status_p_reason": "robot_navi_mode_changed", "status_p": "busy", "status_p_reason": "busy_start_working" } } } }, ...(생략) }</pre>
2. event_job_progress 시작	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "started", "busy_status_reason": "" } } } }, ...(생략) }</pre>
3. event_job_progress 중 use_resource 사용(엘리베이터)	<pre>{ "msg_type": "feedback",</pre>

시작	<pre> "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "use_resource_started", "busy_status_reason": "use_resource_action_started", "target_info": { "action_name": "use_resource", "resource_type": "Teleporter" } } } }, ...(생략) </pre>
4 event_job_progress 중 elevator call	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "use_resource_using", "busy_status_reason": "use_resource_action_using", "target_info": { "action_name": "elevator_call" } } } } }, ...(생략) </pre>
5 event_job_progress 중 elevator 앞 탑승을 위한 entry 지점까지 move 시작	<pre> { "msg_type": "feedback", "msg_body": { </pre>

	<pre> "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "move_started", "busy_status_reason": "move_action_started", "target_info": { "action_name": "move", "waypoint_name": "auto_move_entry_pose", "waypoint_id": null, "finishing_timeout": -1 } } } </pre>
6 event_job_progress 중 elevator 앞 탑승을 위한 entry 지점까지 move 완료	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "move_finished", "busy_status_reason": "move_action_finished", "target_info": { "action_name": "move", "waypoint_name": "auto_move_entry_pose", "waypoint_id": null, "finishing_timeout": -1 } } } } } </pre>

	}
7 event_job_progress 중 elevator 탑승 시작	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "boarding_started", "busy_status_reason": "boarding_action_started", "target_info": { "action_name": "boarding", "resource_type": "Teleporter" } } } }, ...(생략) }</pre>
8 event_job_progress 중 elevator 탑승 완료	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "boarding_finished", "busy_status_reason": "boarding_action_finished", "target_info": { "action_name": "boarding", "resource_type": "Teleporter" } } } }, ...(생략) }</pre>

	}
9 event_job_progress 중 elevator 하차 시작	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "unboarding_started", "busy_status_reason": "unboarding_action_started", "target_info": { "action_name": "unboarding", "resource_type": "Teleporter" } } } }, ...(생략) }</pre>
10 event_map_changed 수신: 층간 이동으로 맵이 변경됨.	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_map_changed", ...(생략) "event_info": { "result": "success" } }, ...(생략) }</pre>
11 event_job_progress 중 elevator 하차 완료	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": {</pre>

	<pre> "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "unboarding_finished", "busy_status_reason": "unboarding_action_finished", "target_info": { "action_name": "unboarding", "resource_type": "Teleporter" } } } }, ...(생략) </pre>
12 event_job_progress 중 use_resource 사용(엘리베이터) 완료	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "use_resource_finished", "busy_status_reason": "use_resource_action_finished", "target_info": { "action_name": "use_resource", "resource_type": "Teleporter" } } } } }, ...(생략) </pre>
13 3 층 3F_front_elv 까지 move 시작	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { </pre>

	<pre> "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "move_started", "busy_status_reason": "move_action_started", "target_info": { "action_name": "move", "waypoint_name": "3F_front_elv", "waypoint_id": "5f4f5814228e35001204bf82", "finishing_timeout": 30 } } }, ...(생략) } </pre>
14 3 층 3F_front_elv 까지 move 완료	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "move_finished", "busy_status_reason": "move_action_finished", "target_info": { "action_name": "move", "waypoint_name": "3F_front_elv", "waypoint_id": "5f4f5814228e35001204bf82", "finishing_timeout": 30 } } } } }, ...(생략) } </pre>
15 event_job_progress 완료	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", </pre>

	<pre> ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c05d57d0b44b002a46a3f9", "busy_status": "finished", "busy_status_reason": "" } } }, ...(생략) </pre>
16 job 완료 이후 worker 의 상태 변경(busy > idle)	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_status_changed", ...(생략) "event_info": { "result": "success", "data": { "prev_status_p": "busy", "prev_status_p_reason": "busy_start_working", "status_p": "idle", "status_p_reason": "idle_done_job_successfully" } } } }, ...(생략) </pre>

6.4. job수행시 세부 feedback event 예시2

- 현재 위치에서 waypoint(3f_303)로 이동 후 wait(30초 동안 huaninput 대기 후) waypoint(3f_304)로 이동하는 경우

설명	Feedback event
1. Job 시작에 대한 worker 의 상태 변경	<pre> { "msg_type": "feedback", "msg_body": { </pre>

	<pre> "name": "event_status_changed", ...(생략) "event_info": { "result": "success", "data": { "prev_status_p": "idle", "prev_status_p_reason": "idle_done_job_successfully", "status_p": "busy", "status_p_reason": "busy_start_working" } } }, ...(생략) </pre>
2. event_job_progress 시작	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c07302d0b44b002a46a442", "busy_status": "started", "busy_status_reason": "" } } } }, ...(생략) </pre>
3 event_job_progress 중 3f_303 까지 move 시작	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c07302d0b44b002a46a442", "busy_status": "move_started", </pre>

	<pre> "busy_status_reason": "move_action_started", "target_info": { "action_name": "move", "waypoint_name": "3F_303", "waypoint_id": "5f4f4e3e228e35001204bf72", "finishing_timeout": -1 } } }, ...(생략) } </pre>
4 event_job_progress 중 3f_303 까지 move 완료	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c07302d0b44b002a46a442", "busy_status": "move_finished", "busy_status_reason": "move_action_finished", "target_info": { "action_name": "move", "waypoint_name": "3F_303", "waypoint_id": "5f4f4e3e228e35001204bf72", "finishing_timeout": -1 } } } } }, ...(생략) } </pre>
5 event_job_progress 중 현재 위치에서 사용자 버튼 입력이 있을 때까지 30 초간 wait 시작	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { </pre>

	<pre> "result": "success", "data": { "job_id": null, "busy_status": "wait_started", "busy_status_reason": "wait_action_started", "target_info": { "action_name": "wait", "wait_type": "human_input", "wait_timeout": 30000 } } }, ...(생략) </pre>
6 event_job_progress 중 현재 위치에서 사용자 버튼 입력이 있을 때까지 30 초간 wait 완료	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": null, "busy_status": "wait_finished", "busy_status_reason": "wait_action_finished", "target_info": { "action_name": "wait", "wait_type": "human_input", "wait_timeout": 30000 } } } } }, ...(생략) </pre>
7 event_job_progress 중 3f_304 까지 move 시작	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", </pre>

	<pre> ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c07302d0b44b002a46a442", "busy_status": "move_started", "busy_status_reason": "move_action_started", "target_info": { "action_name": "move", "waypoint_name": "3F_304", "waypoint_id": "5f4f4f49228e35001204bf78", "finishing_timeout": -1 } } }, ...(생략) </pre>
8 event_job_progress 중 3f_304 까지 move 완료	<pre> { "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c07302d0b44b002a46a442", "busy_status": "move_finished", "busy_status_reason": "move_action_finished", "target_info": { "action_name": "move", "waypoint_name": "3F_304", "waypoint_id": "5f4f4f49228e35001204bf78", "finishing_timeout": -1 } } } }, ...(생략) } </pre>

9 event_job_progress 완료	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_job_progress", ...(생략) "event_info": { "result": "success", "data": { "job_id": "60c07302d0b44b002a46a442", "busy_status": "finished", "busy_status_reason": "" } } }, ...(생략) }</pre>
10 job 완료 이후 worker 의 상태 변경(busy > idle)	<pre>{ "msg_type": "feedback", "msg_body": { "name": "event_status_changed", ...(생략) "event_info": { "result": "success", "data": { "prev_status_p": "busy", "prev_status_p_reason": "busy_start_working", "status_p": "idle", "status_p_reason": "idle_done_job_successfully" } } }, ...(생략) }</pre>

7. 기타 정보

7.

7.1. worker_info와 map_info 활용 방법

Worker_info 에는 robot 의 위치 정보(x, y, theta)가 포함됩니다. x, y 정보는 실제 거리 좌표계(meter 단위)에서 원점(맵을 작성 시작한 위치) 기준으로의 상대 좌표입니다.



YUJIN ROBOT | INNOVATION FOR A BETTER WORLD

Address: Harmony-ro 187 Beon-gil 33, Yeonsu-gu, Incheon (22013)

Phone: +82-32-550-2334

Fax: +82-32-550-2301

E-mail: sales@yujinrobot.com

www.yujinrobot.com